

Bases de Datos

Clase 12: Evaluación de consultas

Páginas, disco y buffer | Iterador lineal | Operadores lineales

Nested Loop Join | Block Nested Loop Join | External Merge Sort

Contenidos revisados

- Modelo relacional
- Consultas en álgebra relacional
- Modelamiento con el modelo E/R
- Transformación de un MER a un esquema relacional
- Dependencias funcionales y anomalías
- Formas normales 1, 2, 3 y BC → **Proyecto E1**
- SQL → **Proyecto E2**
- Vistas, triggers y stored procedures
- Transacciones, ACID, schedules y locks
- Recuperación ante fallas
- Índices

Páginas, disco y buffer

Repaso

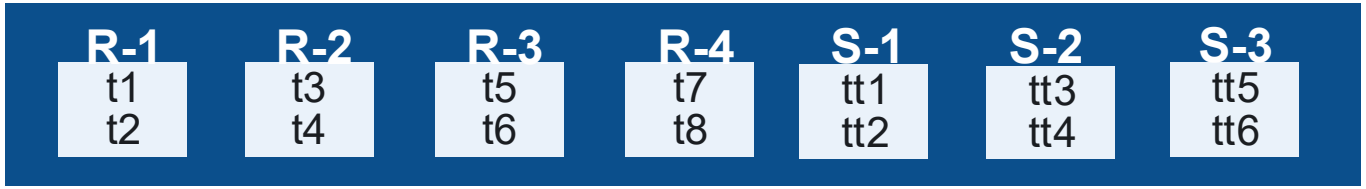
Para operar con las tuplas de una relación, la base de datos carga desde el disco la página que contiene dichas tuplas.

Para cargar estas páginas, la base de datos reserva un espacio en RAM denominado **buffer**.



Páginas, disco y buffer

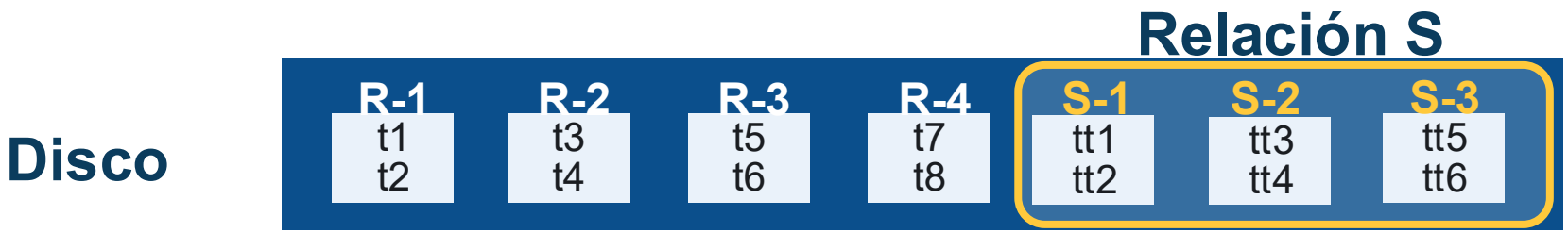
Disco



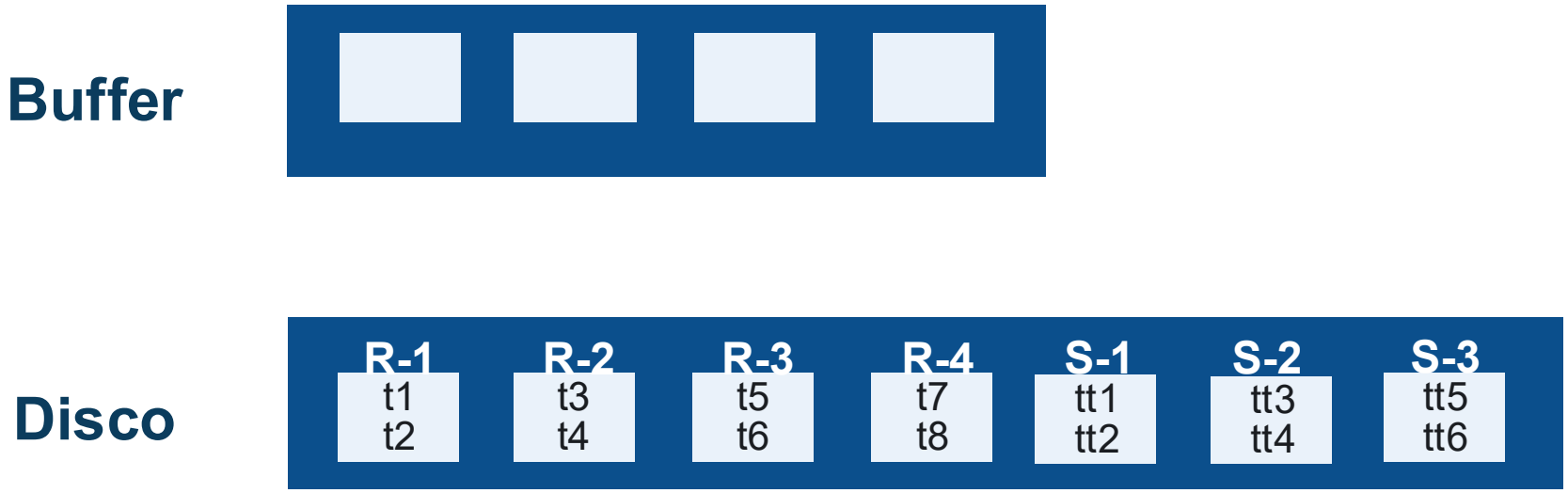
Páginas, disco y buffer



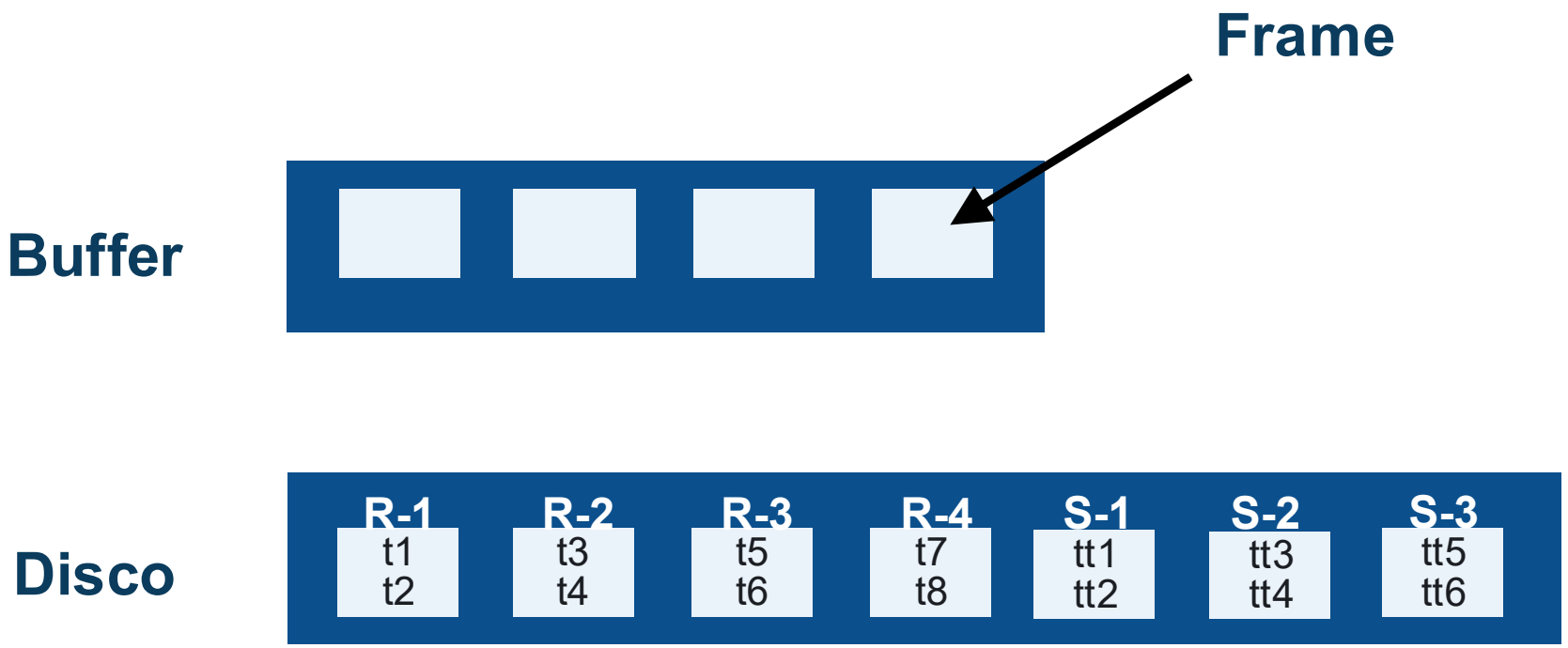
Páginas, disco y buffer



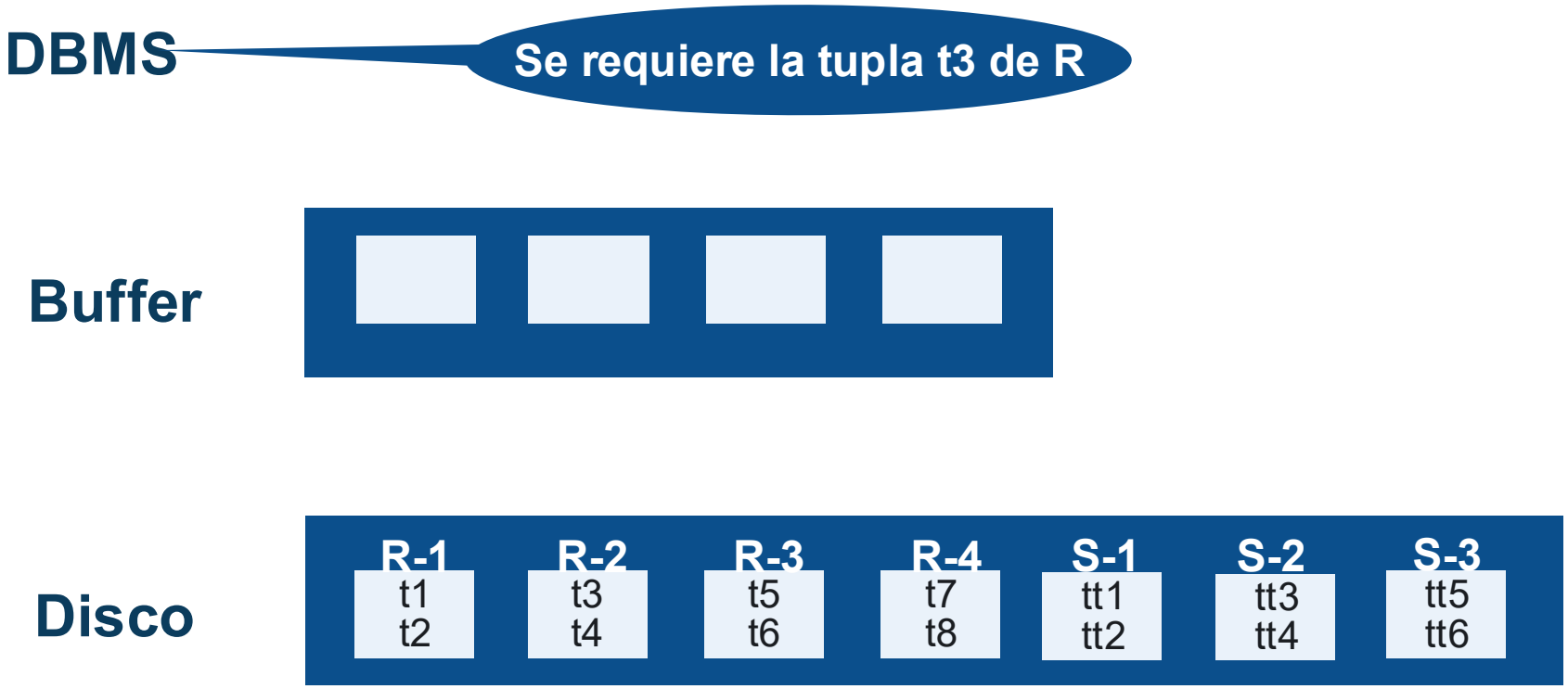
Páginas, disco y buffer



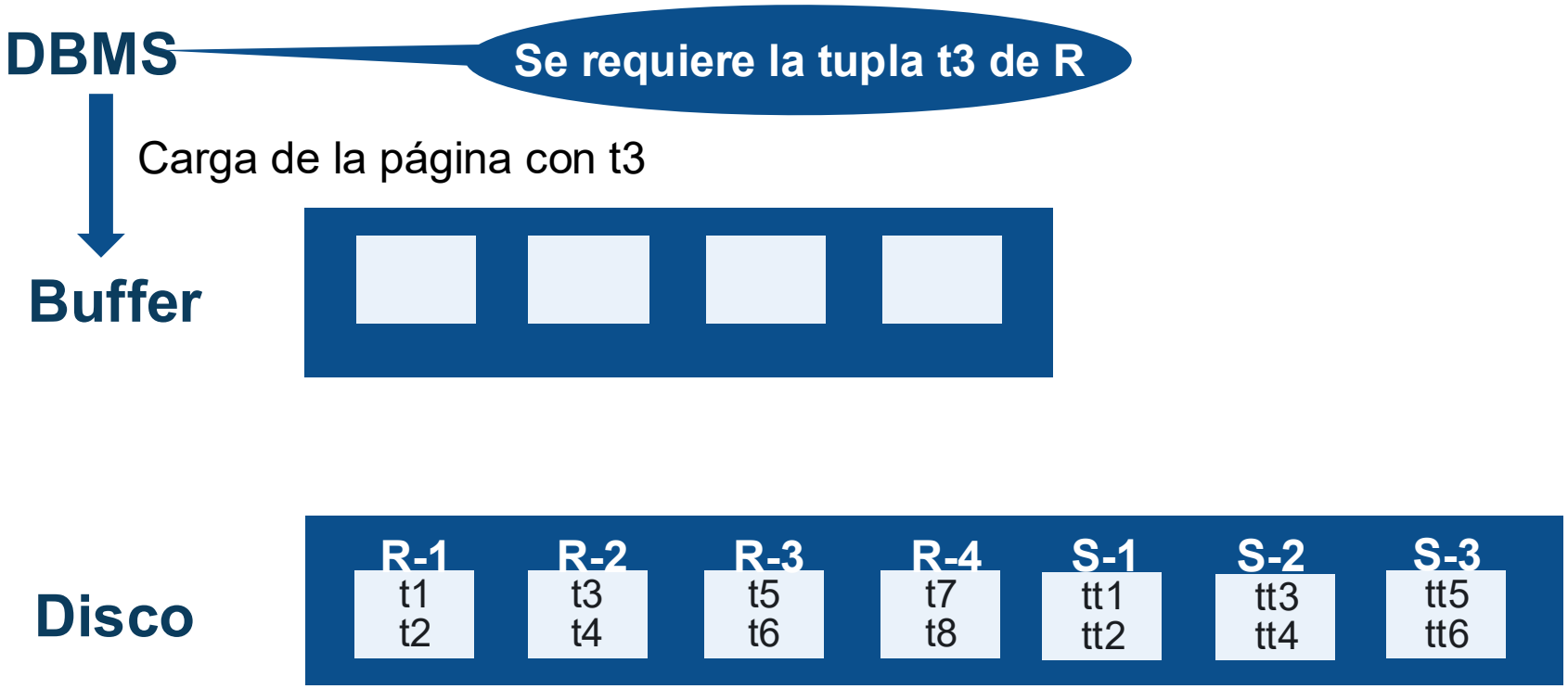
Páginas, disco y buffer



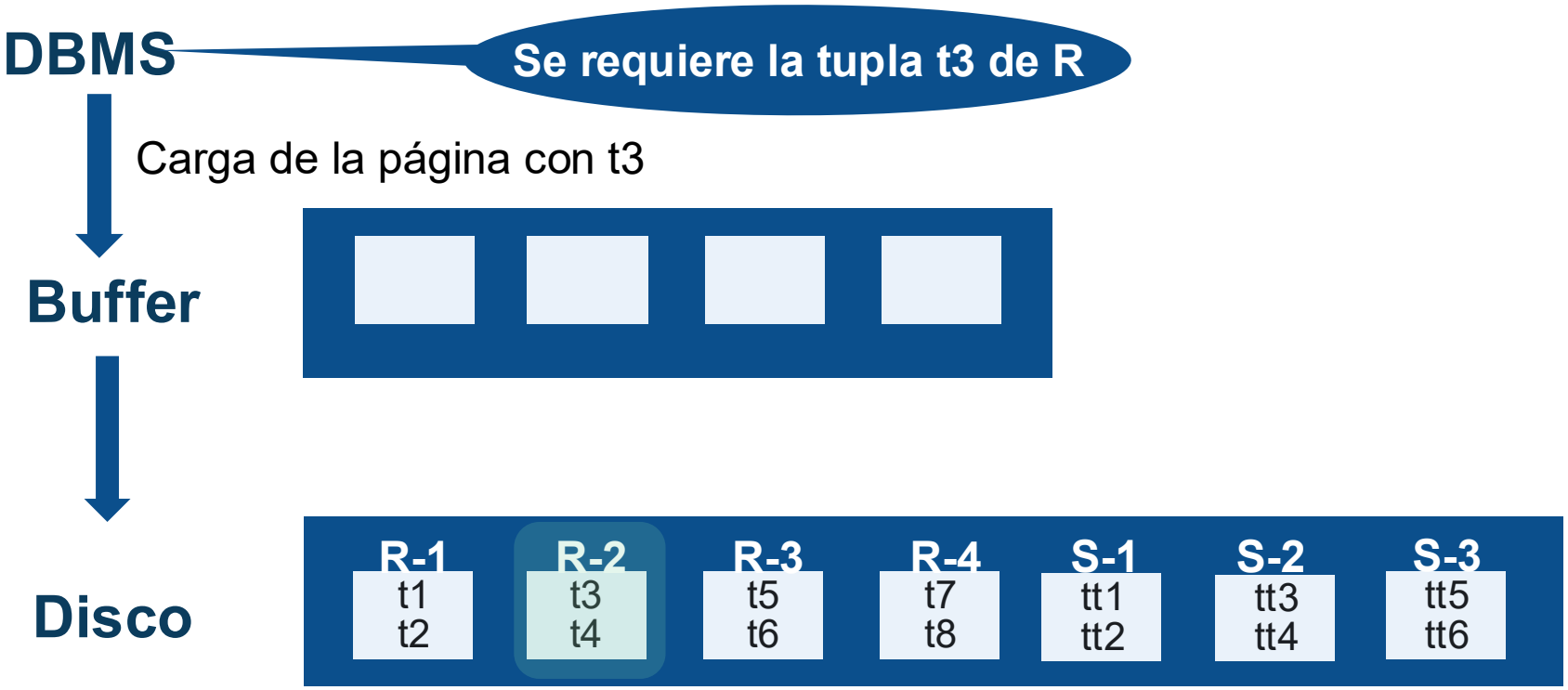
Páginas, disco y buffer



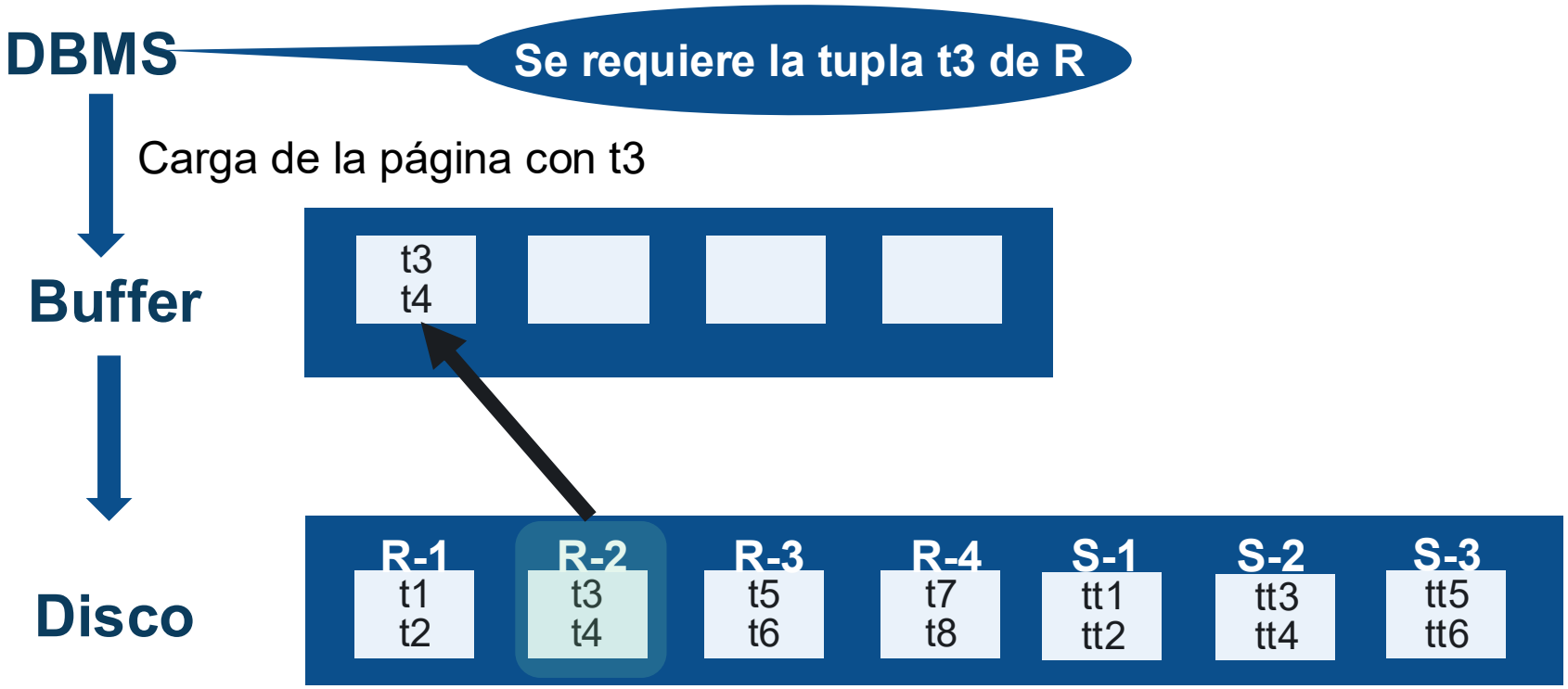
Páginas, disco y buffer



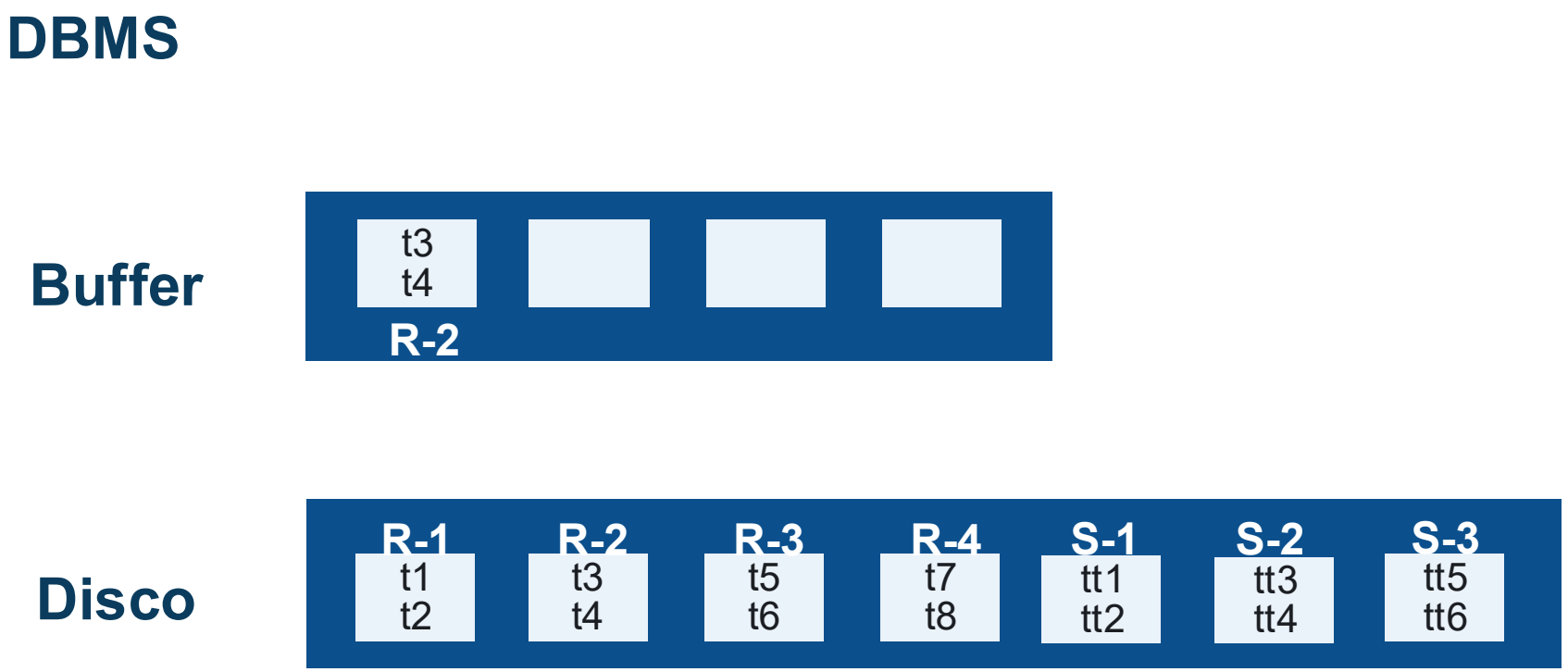
Páginas, disco y buffer



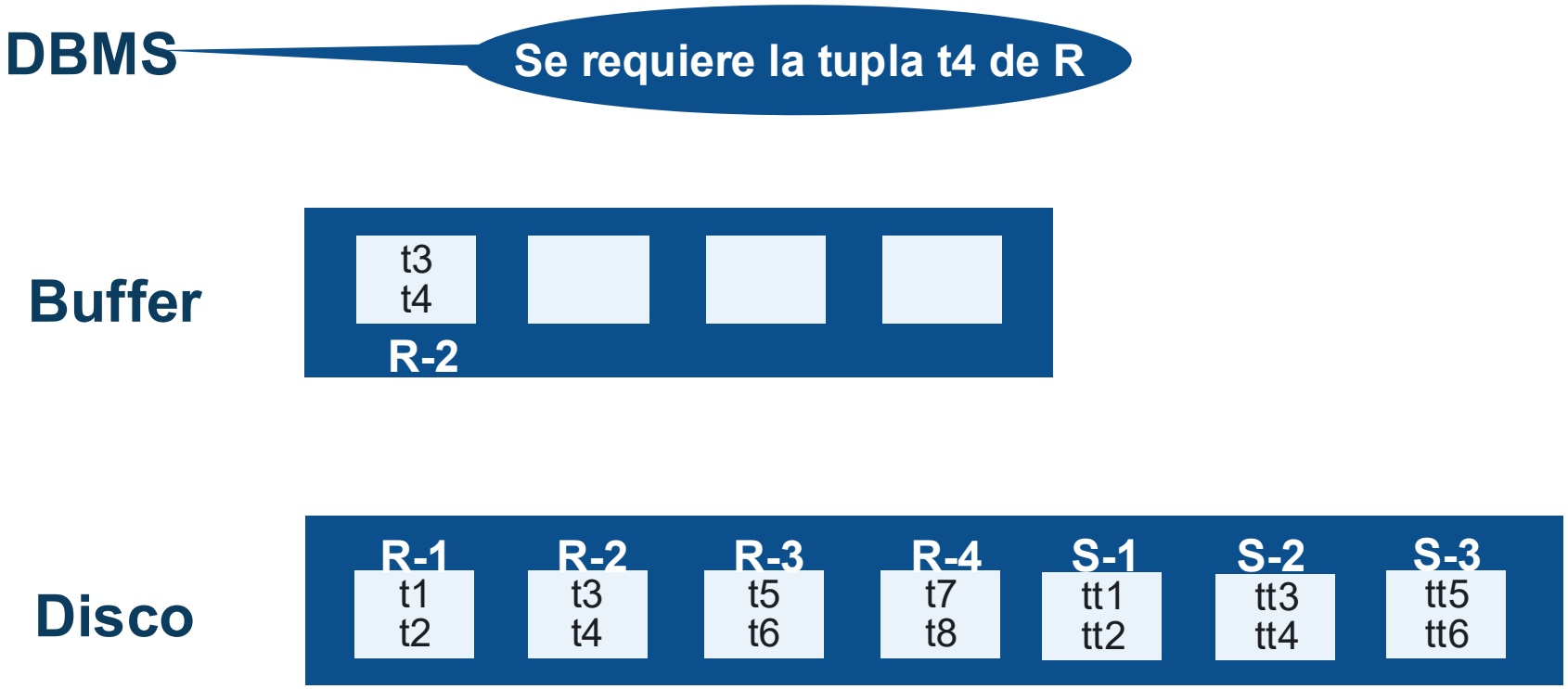
Páginas, disco y buffer



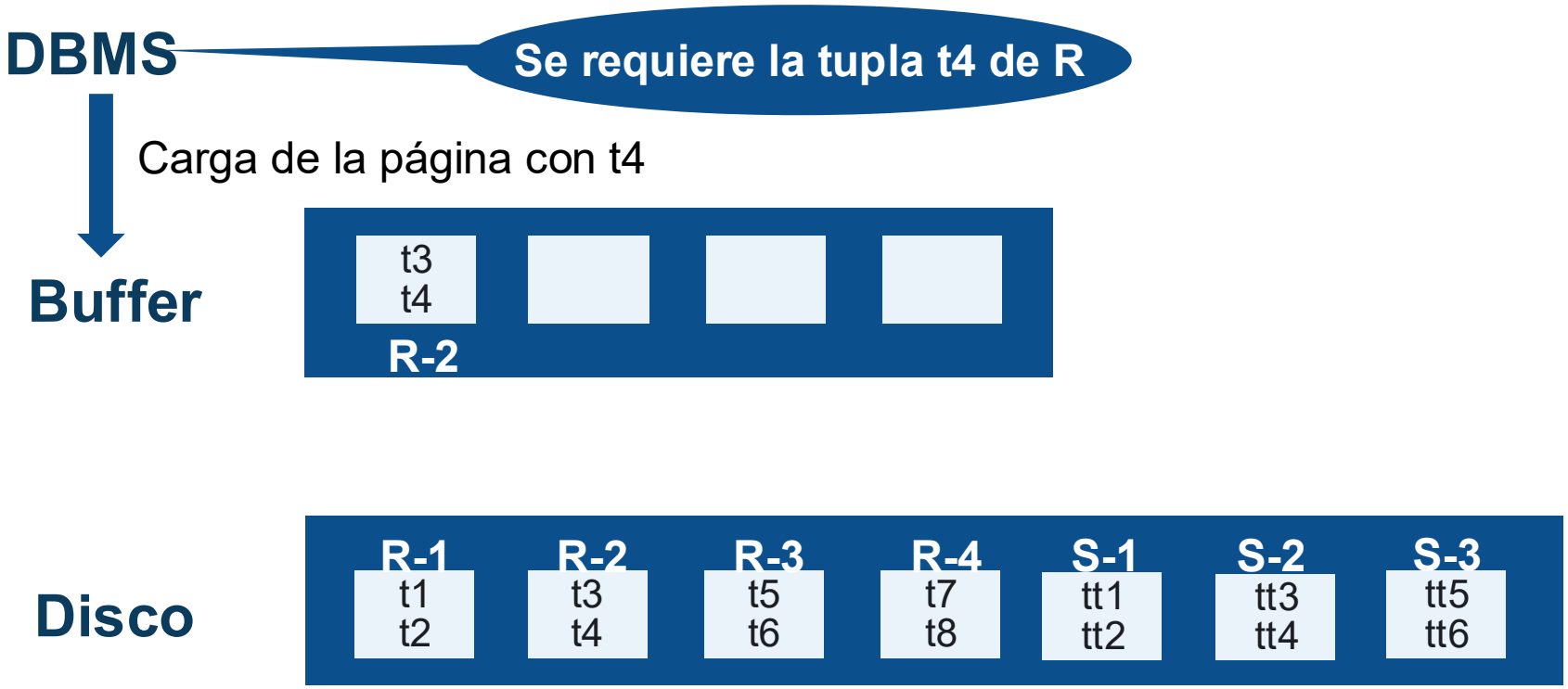
Páginas, disco y buffer



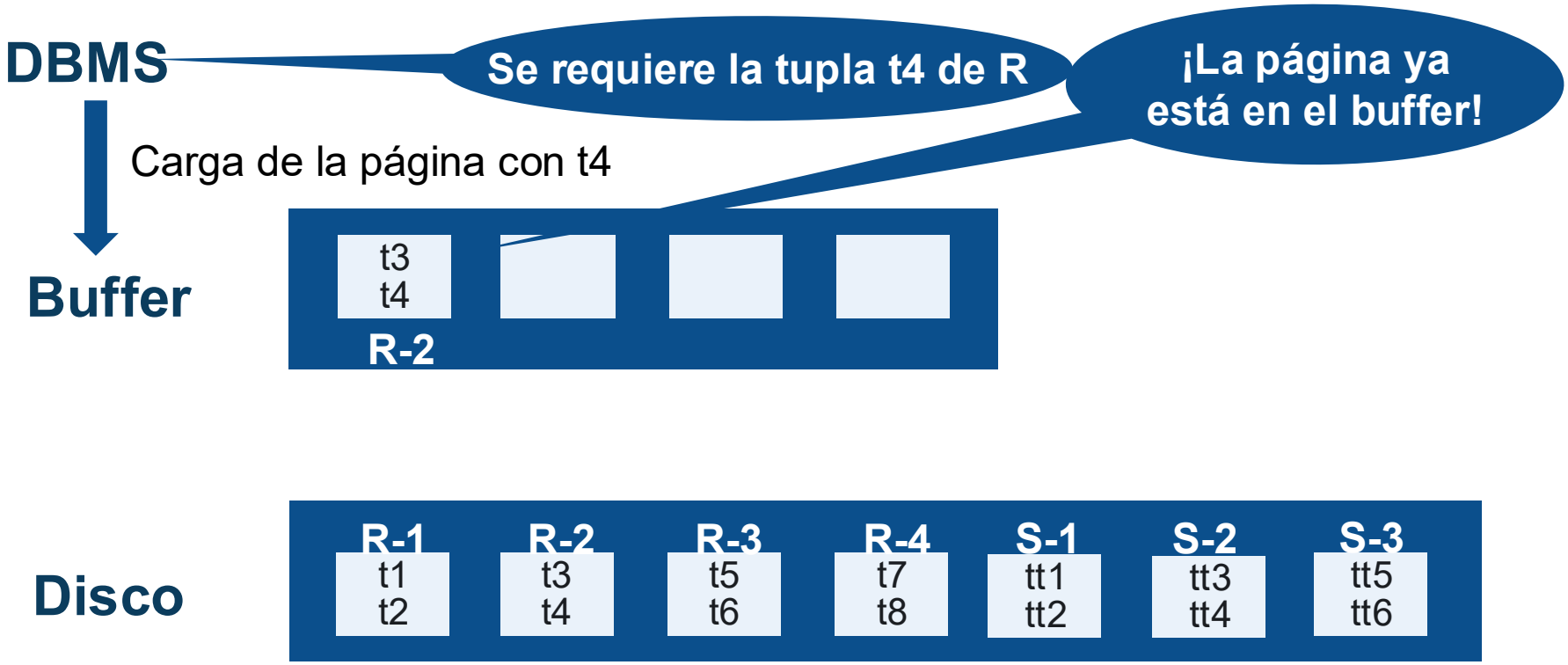
Páginas, disco y buffer



Páginas, disco y buffer



Páginas, disco y buffer



Costo de un algoritmo



¿Cuántas veces es necesario leer una página desde el disco, o escribir una página en él?

Las operaciones en buffer (RAM) son órdenes de magnitud más rápidas que la lectura/escritura en disco: se les asigna **costo 0**.



I/O

Lectura/escritura de páginas en disco: costo considerado



0

Operaciones en buffer (RAM): costo

Algoritmos en una BD

Los algoritmos implementan la interfaz de un **iterador lineal**:

`open()` `next()` `close()`

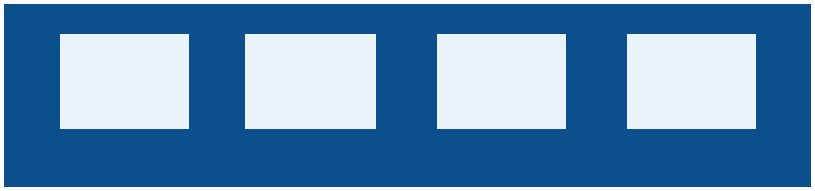
Para una relación R:

Método	Comportamiento
`R.open()`	Se posiciona **antes** de la primera tupla de R
`R.next()`	Retorna la siguiente tupla o NULL
`R.close()`	Cierra el iterador

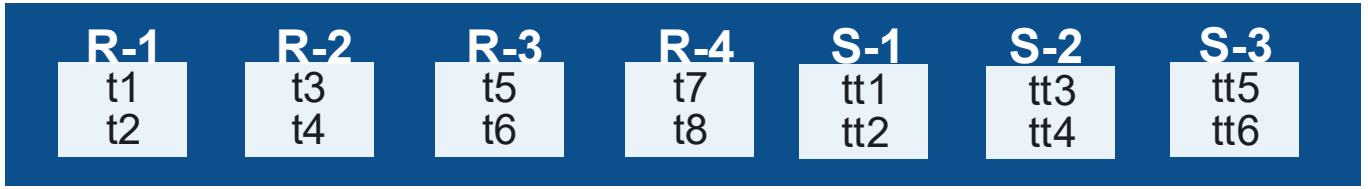
Páginas, disco y buffer

DBMS

Buffer



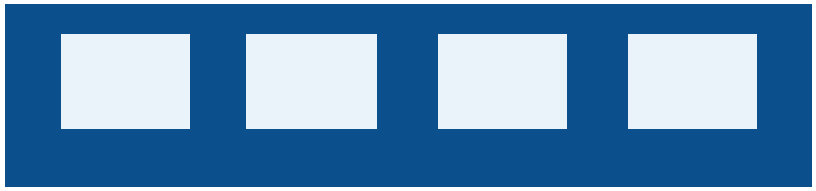
Disco



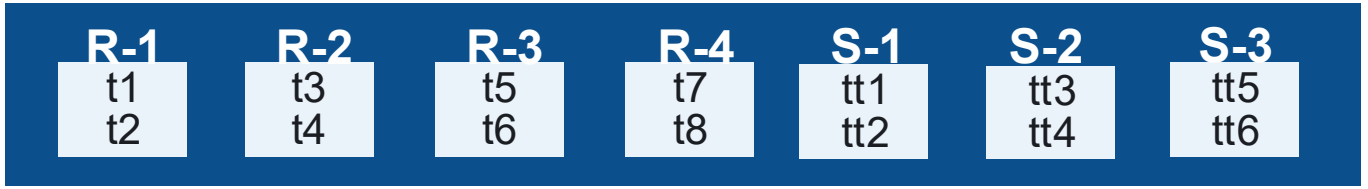
Páginas, disco y buffer

DBMS R.open()

Buffer



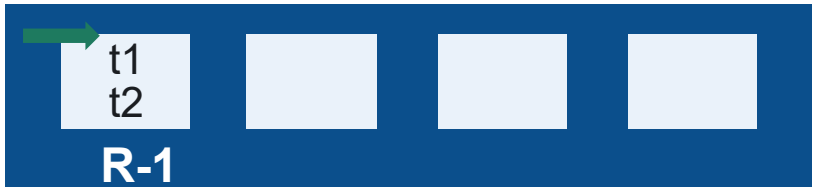
Disco



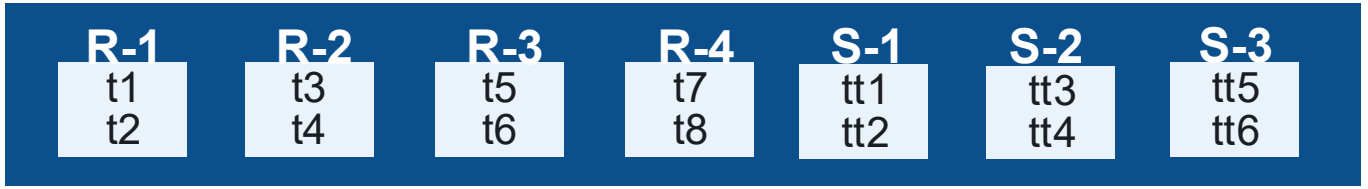
Páginas, disco y buffer

DBMS R.open()

Buffer



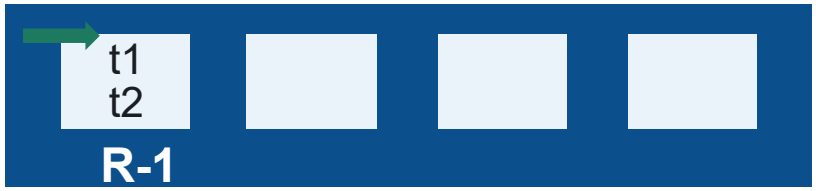
Disco



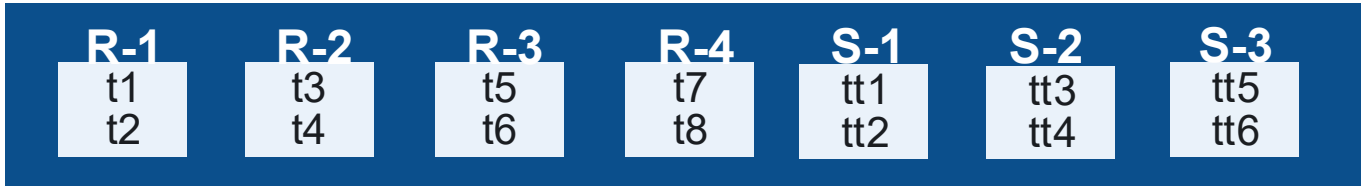
Páginas, disco y buffer

DBMS R.next()

Buffer



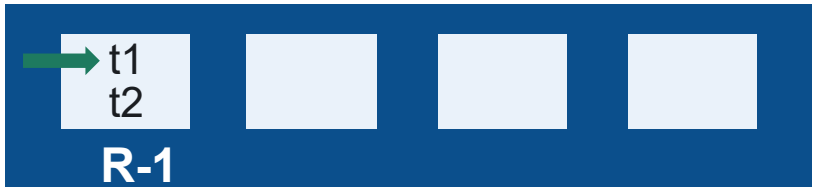
Disco



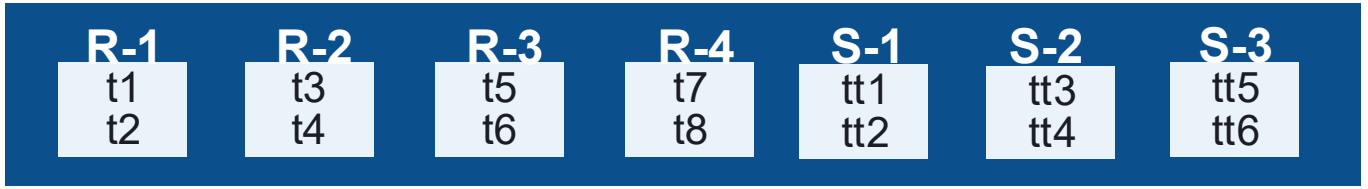
Páginas, disco y buffer

DBMS R.next()

Buffer



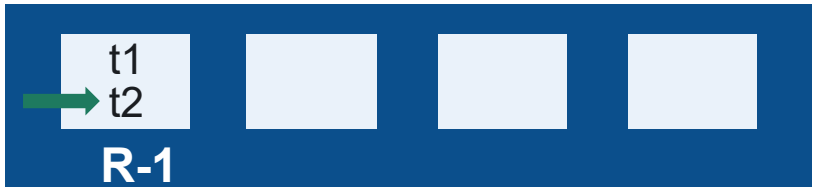
Disco



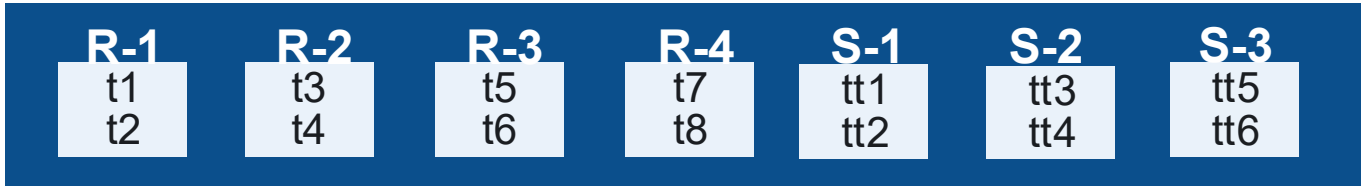
Páginas, disco y buffer

DBMS R.next()

Buffer



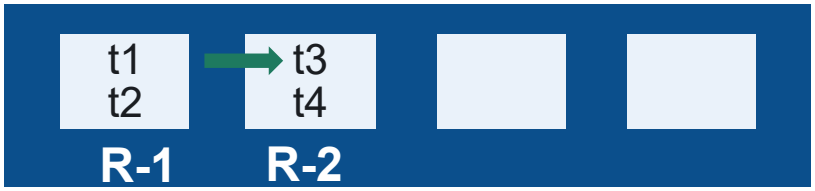
Disco



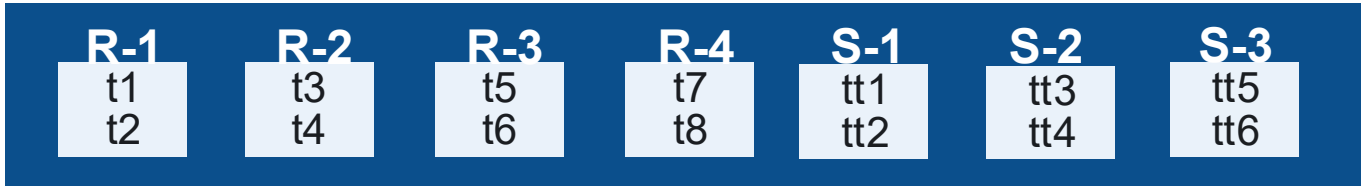
Páginas, disco y buffer

DBMS R.next()

Buffer



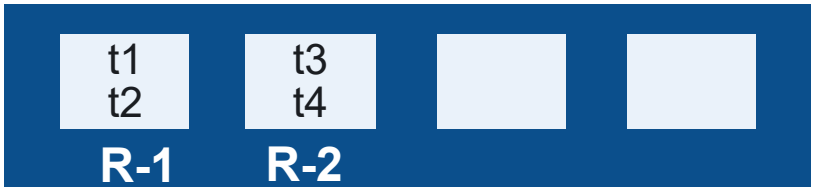
Disco



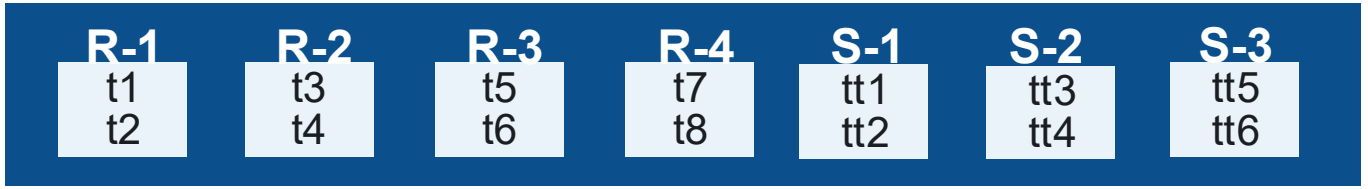
Páginas, disco y buffer

DBMS R.close()

Buffer



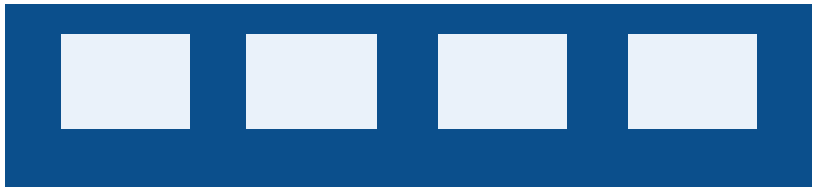
Disco



SELECT * FROM R

DBMS

Buffer



```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

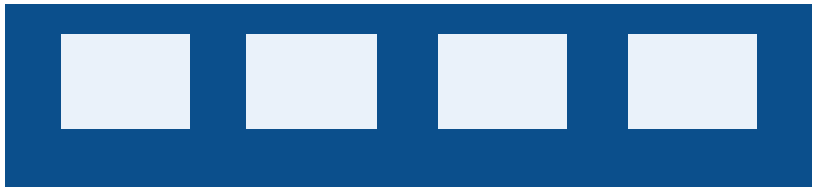
Disco



SELECT * FROM R

DBMS

Buffer



```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

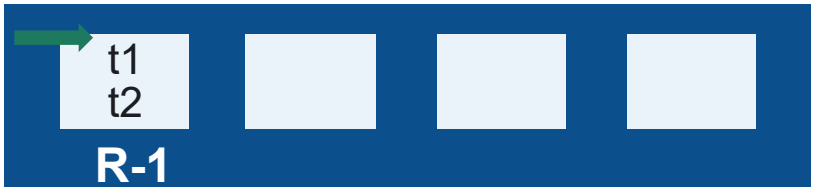
Disco



SELECT * FROM R

DBMS

Buffer



Disco




```
R.open()

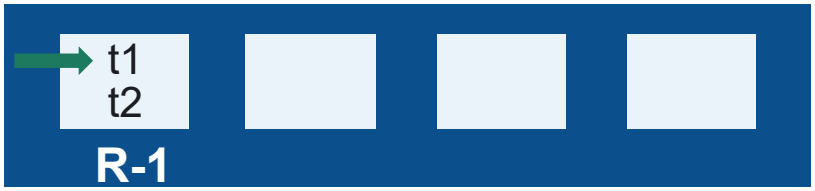
 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

SELECT * FROM R

DBMS

Buffer




```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

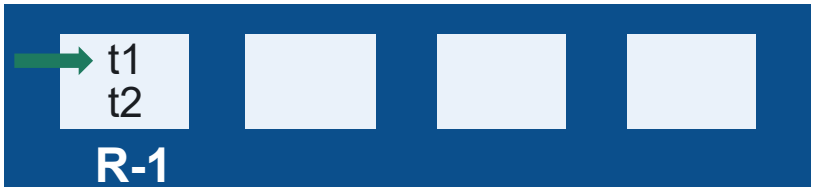
Disco



SELECT * FROM R

DBMS

Buffer

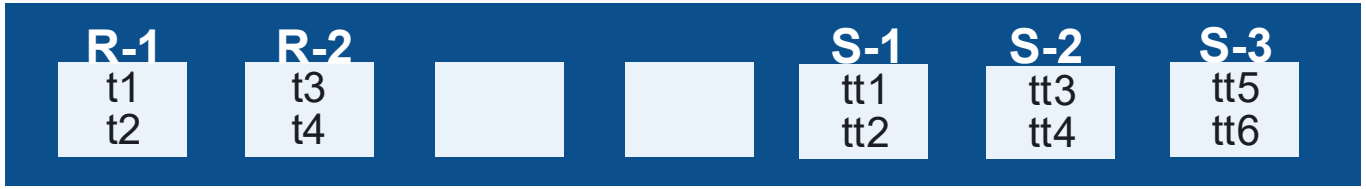



```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

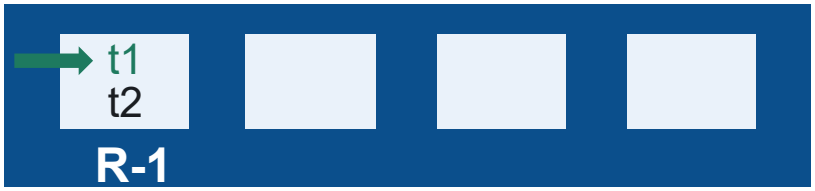
Disco



SELECT * FROM R

DBMS

Buffer

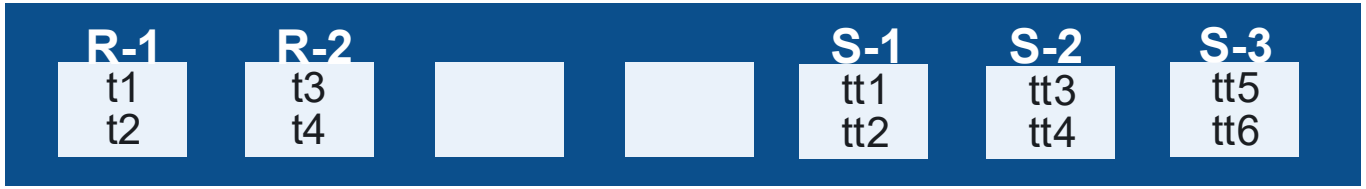



```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

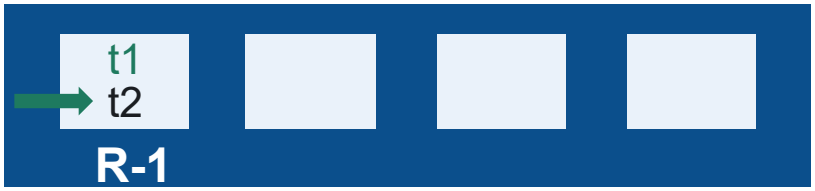
Disco



SELECT * FROM R

DBMS

Buffer




```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

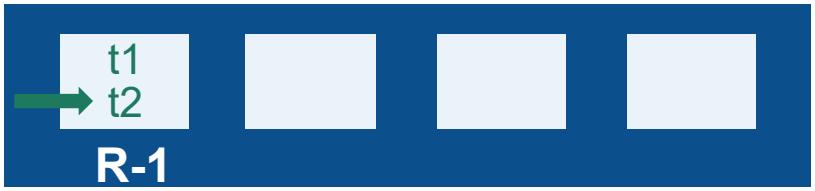
Disco



SELECT * FROM R

DBMS

Buffer




```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

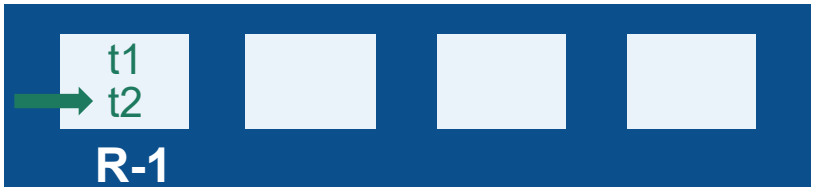
Disco



SELECT * FROM R

DBMS

Buffer




```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

Disco

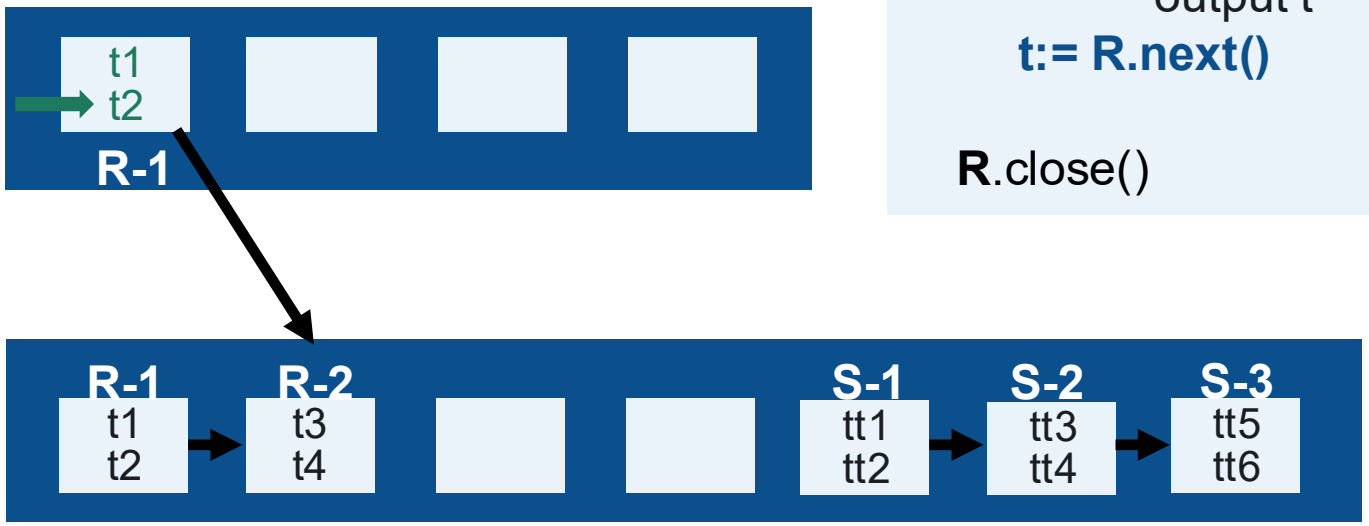


SELECT * FROM R

DBMS

Buffer

Disco




```
R.open()

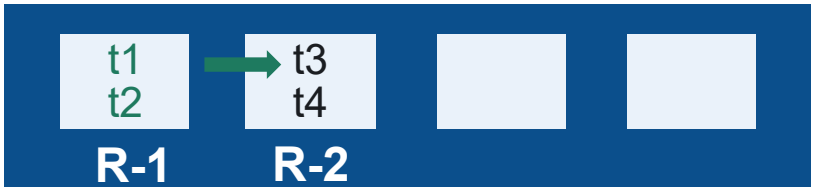
 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

SELECT * FROM R

DBMS

Buffer




```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

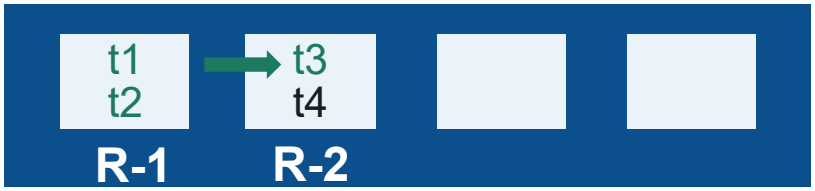
Disco



SELECT * FROM R

DBMS

Buffer




```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

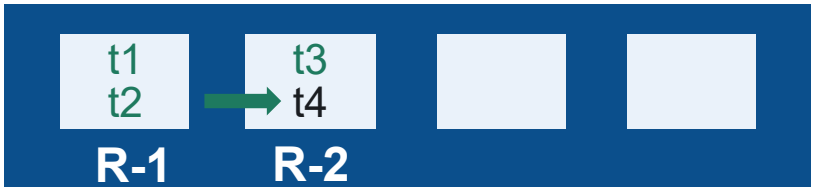
Disco



SELECT * FROM R

DBMS

Buffer




```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

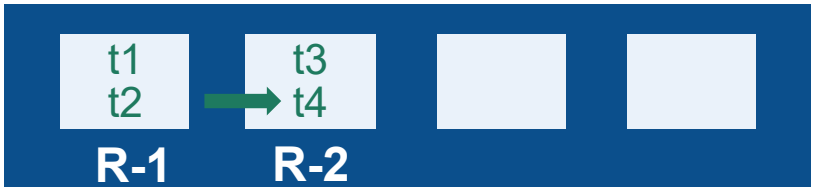
Disco



SELECT * FROM R

DBMS

Buffer

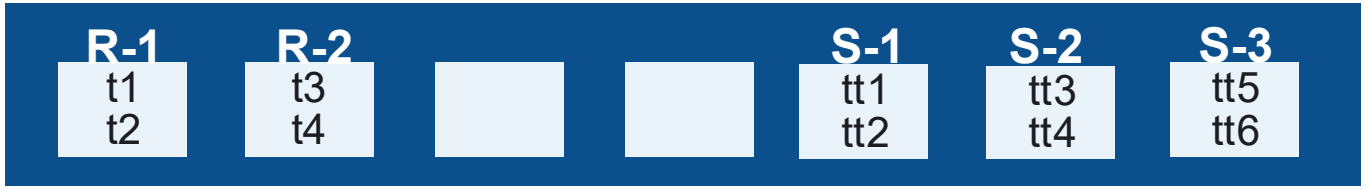



```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

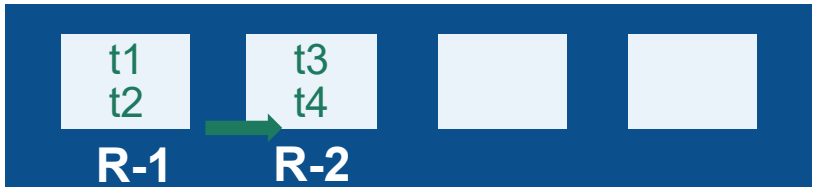
Disco



SELECT * FROM R

DBMS

Buffer




```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

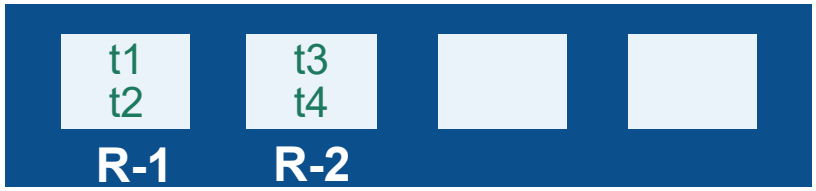
Disco



SELECT * FROM R

DBMS

Buffer




```
R.open()

 t:= R.next()
while t != null do
            output t
   t:= R.next()

R.close()
```

Disco



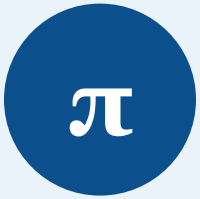
Operadores como iteradores

Cada operador del álgebra relacional implementa la interfaz de un **iterador lineal**:



Selección

```
open()
next()
close()
```



Proyección

```
open()
next()
close()
```



Join

```
open()
next()
close()
```



Operadores lineales

Selección y proyección

Selección



El algoritmo de selección varía según si la consulta es de igualdad (=) o de rango (<, >).



También depende de si el atributo de selección está indexado.



Implementa la interfaz de **iterador lineal**.

Selección

Sin índice

Selección sobre una tabla **R**:

```
open()
  R.open()

close()
  R.close()
```

```
next() // siguiente seleccionado
  t := R.next()
  while t != null do
    if t satisface condición then
      return t
    t := R.next()
  return null
```



Sin índice, necesariamente se recorre toda la relación R: **FULL SCAN**.

Selección

Sin índice

Selección sobre una tabla R

```
open()
  R.open()
next() // siguiente seleccionado
  t := R.next()
  while t != null do
    if t satisface condición then
      return t
    t := R.next()
  return null
close()
  R.close()
```

Recorrido de Sel = $\sigma_{cond}(R)$

```
Sel.open()

t := Sel.next()
while t != null do
  output t
  t := Sel.next()

Sel.close()
```

Selección

Con índice y consulta de igualdad

Selección sobre una tabla **R** por un atributo indexado con un índice **I**:

```
open()
  I.open()
  I.search(Atributo = valor)
next()
  t := I.next()
  while t != null do
    return t
  return null
close()
  I.close()
```



Acceso mediante el índice **I**

Selección

Con índice y consulta de igualdad



Solo se leen las páginas que satisfacen la condición (más I/O si muchas tuplas la satisfacen).



El costo cambia si el índice es **clustered** o **unclustered** (¿por qué?).



Si el atributo es llave primaria, la operación tiene prácticamente un costo de **I/O ≈ 1**.

Selección

¿Cómo realizar este tipo de consultas de forma eficiente?



Usando Proyección

Proyección

Algoritmo muy sencillo

```
open()
  R.open()
next()
  t := R.next()
  while t != null do
    return project(t, atributos)
  return null
close()
  R.close()
```



Necesariamente se recorre toda la relación **R**.

Joins



Costo

Operación muy costosa.



Supuesto

Solo se considerarán restricciones de igualdad, por ejemplo:

```
R.a = S.a
```

Nested Loop Join

Se desea calcular el join entre **R** y **S** cuando se satisface un predicado **p**.

```
open()
  R.open()
  S.open()
  r := R.next()
```

```
close()
  R.close()
  S.close()
```

$$R \bowtie_p S$$

Nested Loop Join

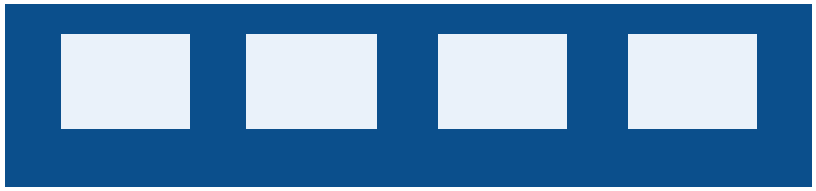
Se desea calcular el join entre **R** y **S** cuando se satisface un predicado **p**.

```
next()
  while r != null do
    s := S.next()
    if s == null then
      S.close()
      r := R.next()
      S.open()
    else if (r, s) satisfacen p then
      return (r, s)
  return null
```

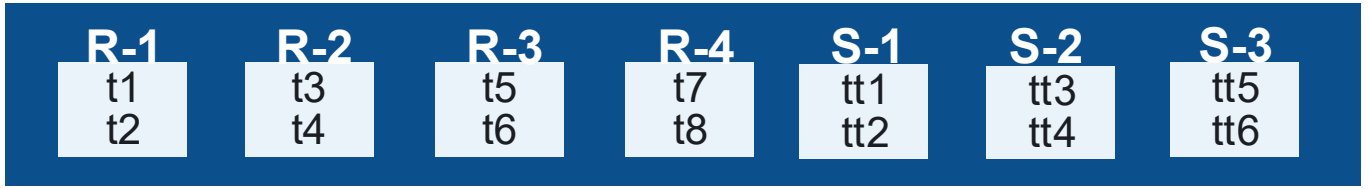
Nested Loop Join

DBMS

Buffer



Disco



Join = $R \bowtie_p S$

```
Join.open()

 t:= Join.next()
while t != null do
            output t
   t:= Join.next()

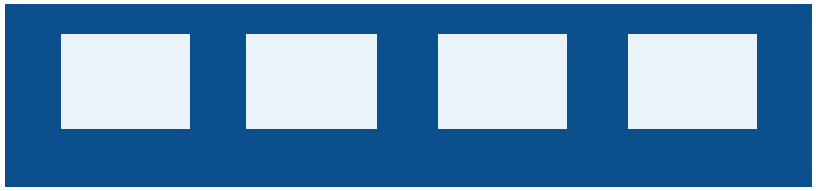
Join.close()
```

Nested Loop Join

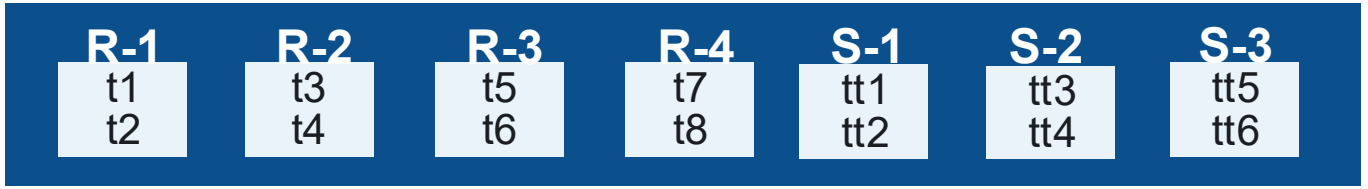
DBMS $R \bowtie_p S$ …Join.open()

```
R.open()
S.open()
r:= R.next()
```

Buffer



Disco

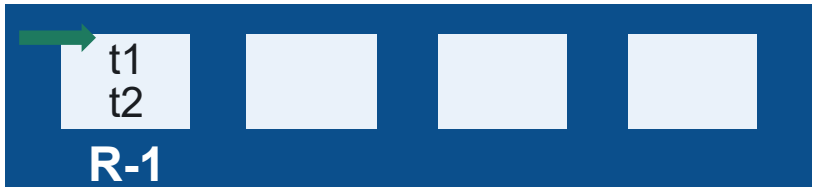


Nested Loop Join

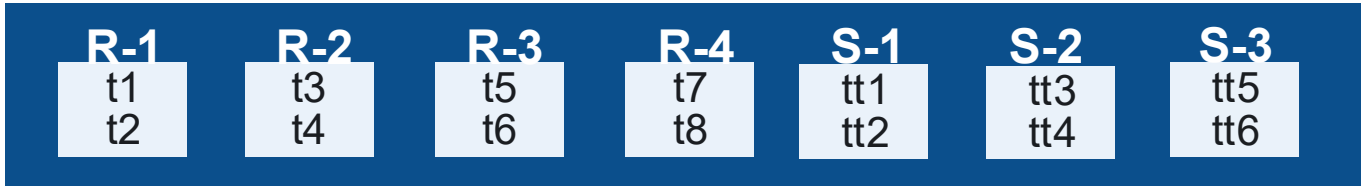
DBMS $R \bowtie_p S$ …Join.open()

```
R.open()
S.open()
r:= R.next()
```

Buffer



Disco

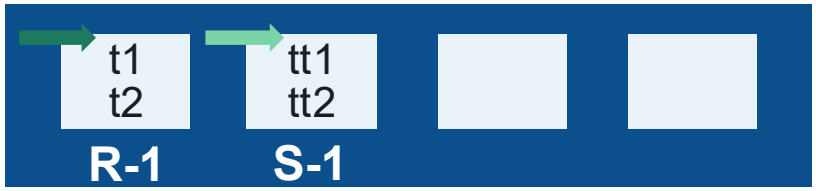


Nested Loop Join

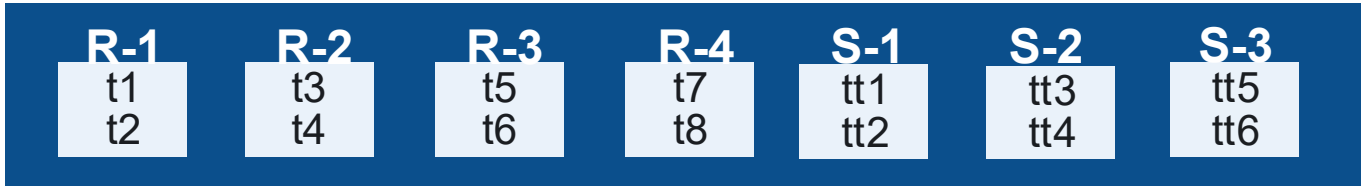
DBMS $R \bowtie_p S$ …Join.open()

```
R.open()
S.open()
r:= R.next()
```

Buffer



Disco

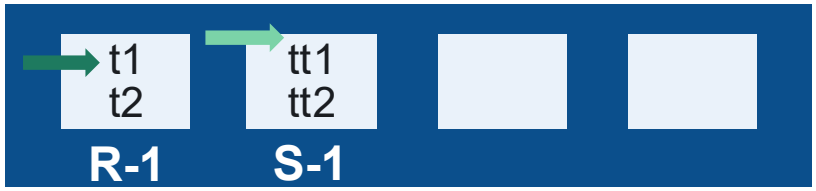


Nested Loop Join

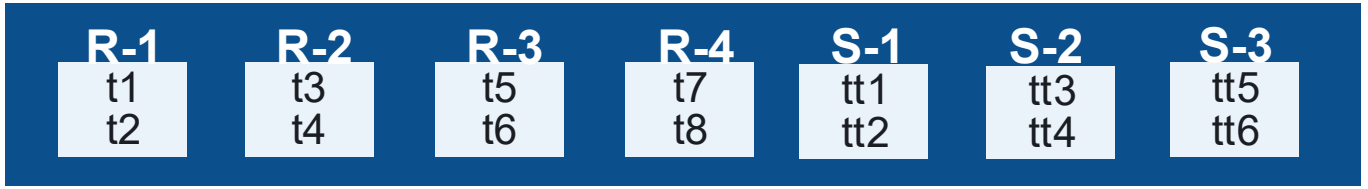
DBMS $R \bowtie_p S$ …Join.open()

```
R.open()
S.open()
r:= R.next()
```

Buffer



Disco

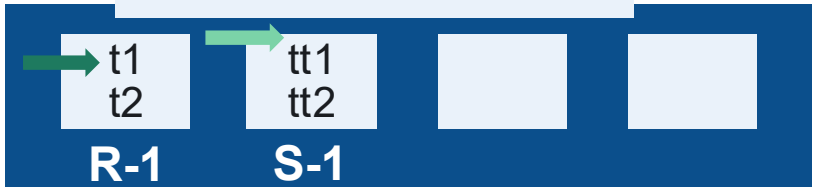


Nested Loop Join

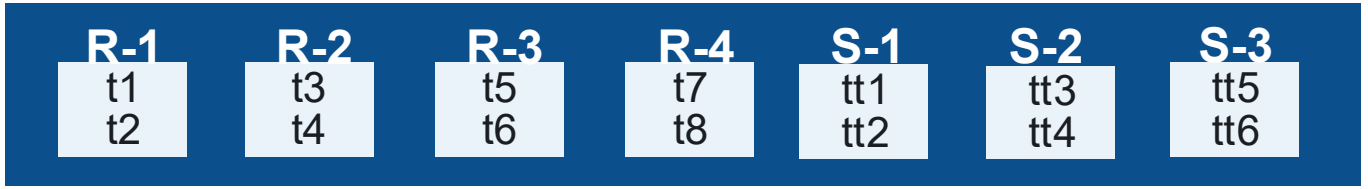
DBMS

```
t:= Join.next()
while t != null do
        output t
    t:= Join.next()
```

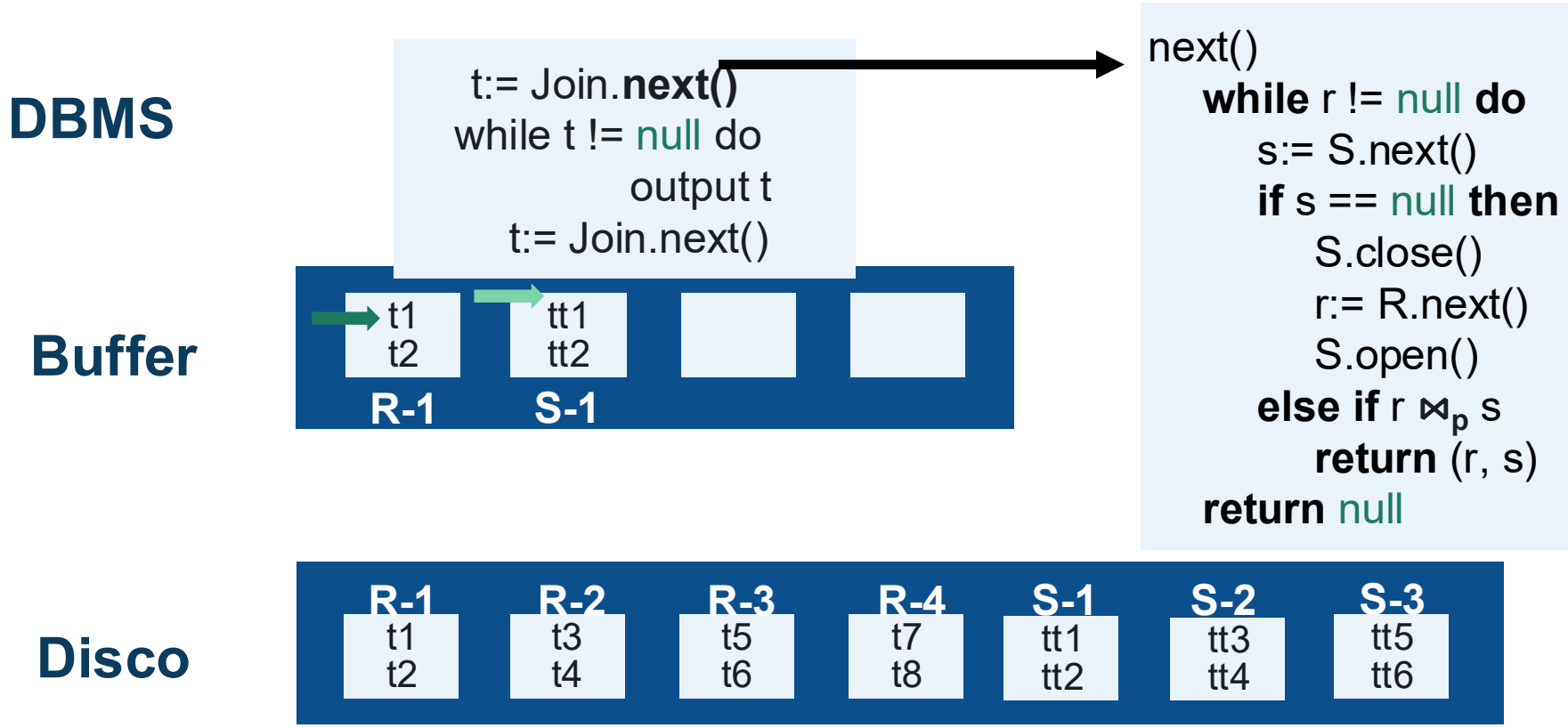
Buffer



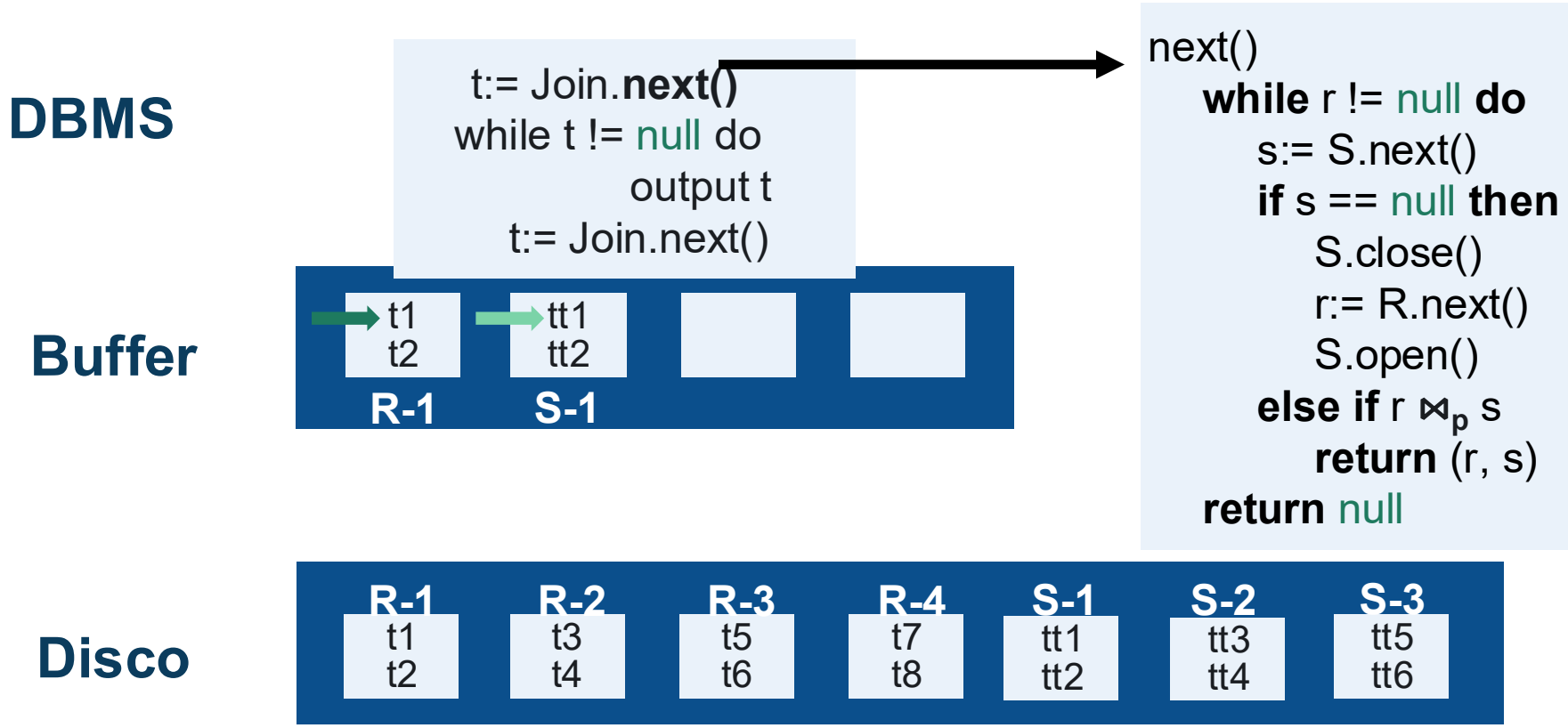
Disco



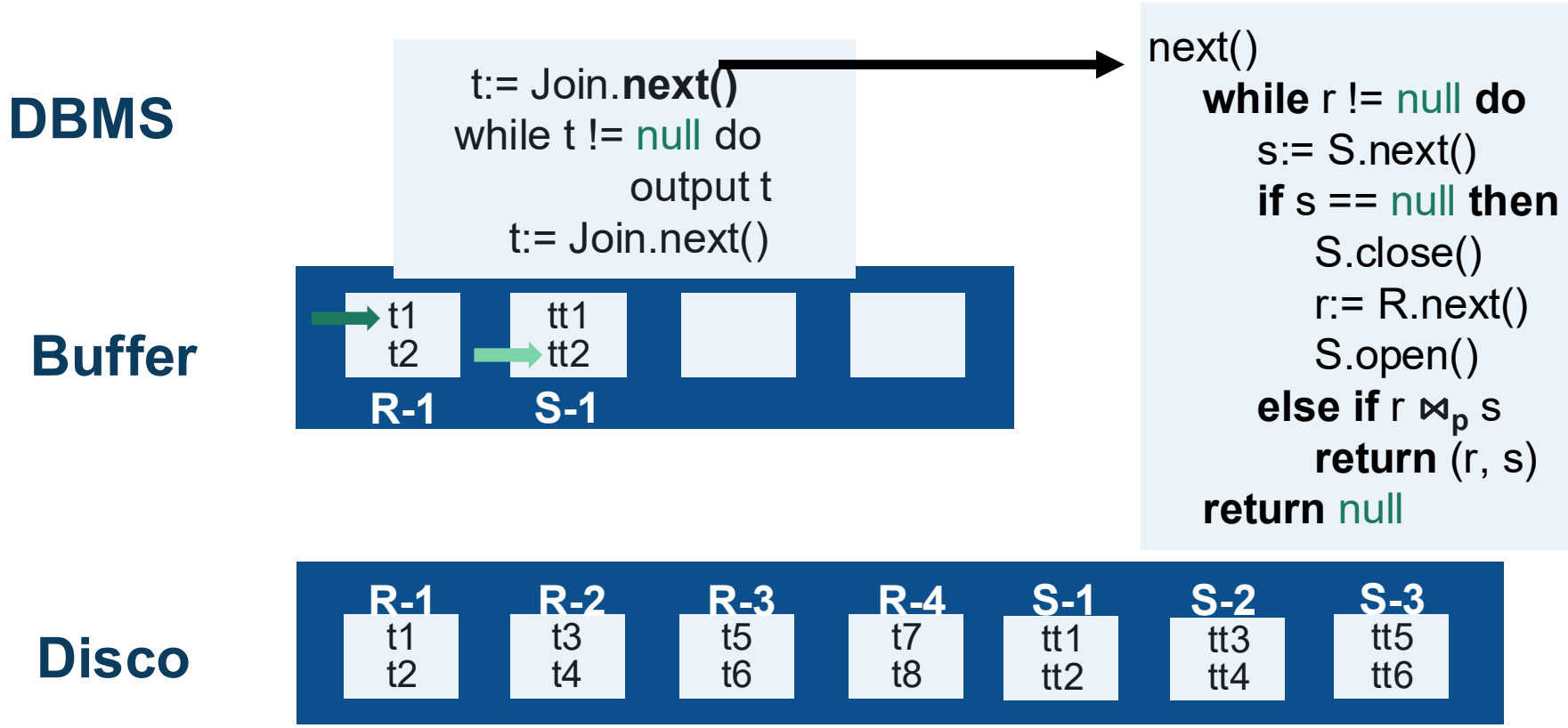
Nested Loop Join



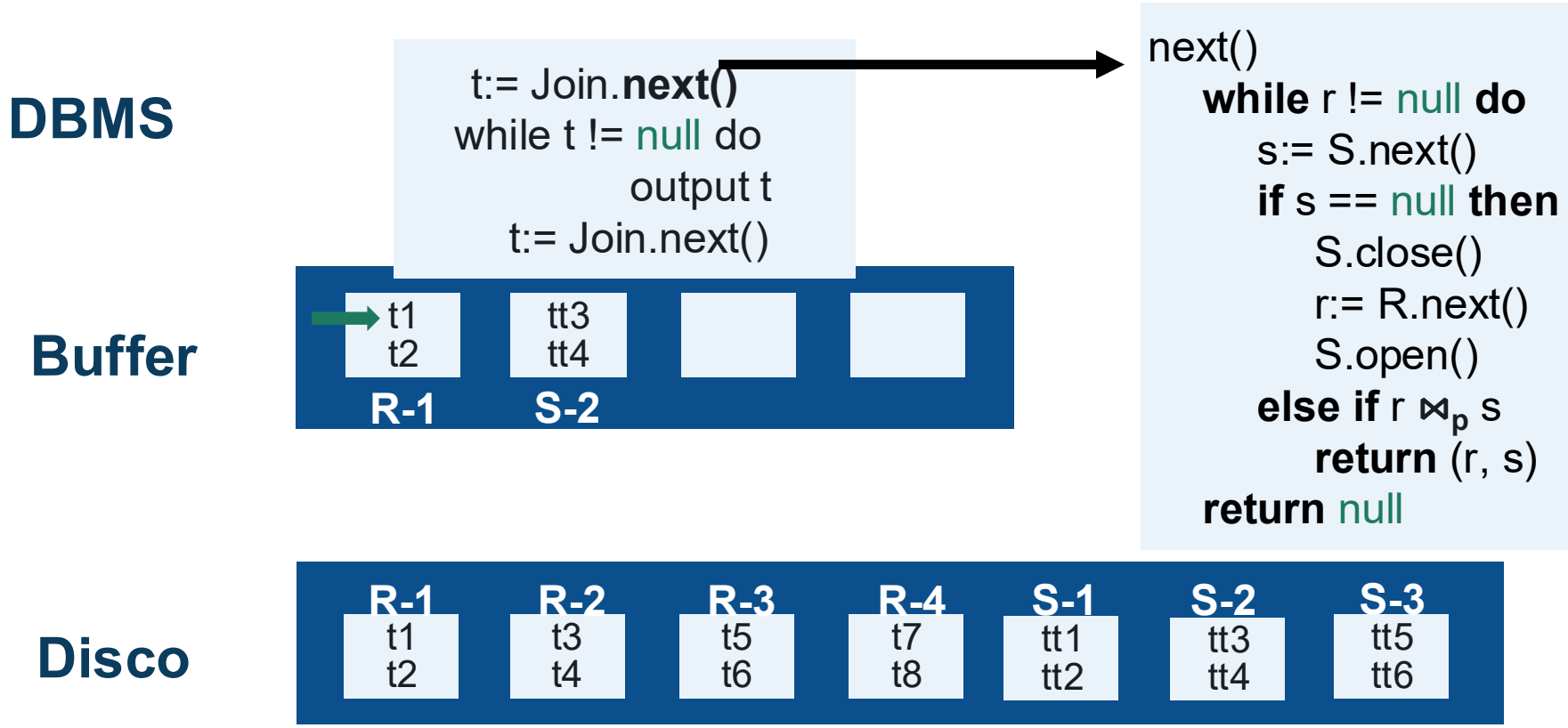
Nested Loop Join



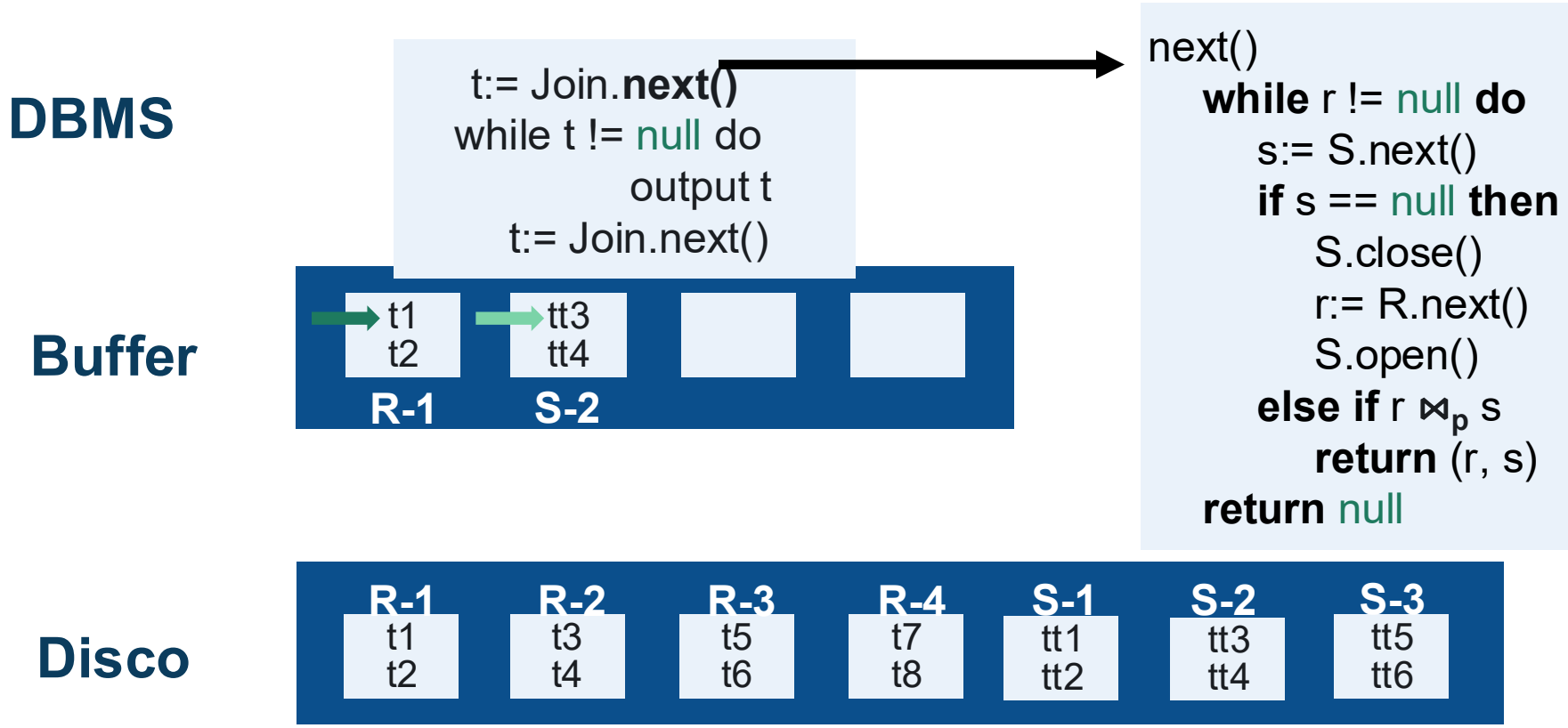
Nested Loop Join



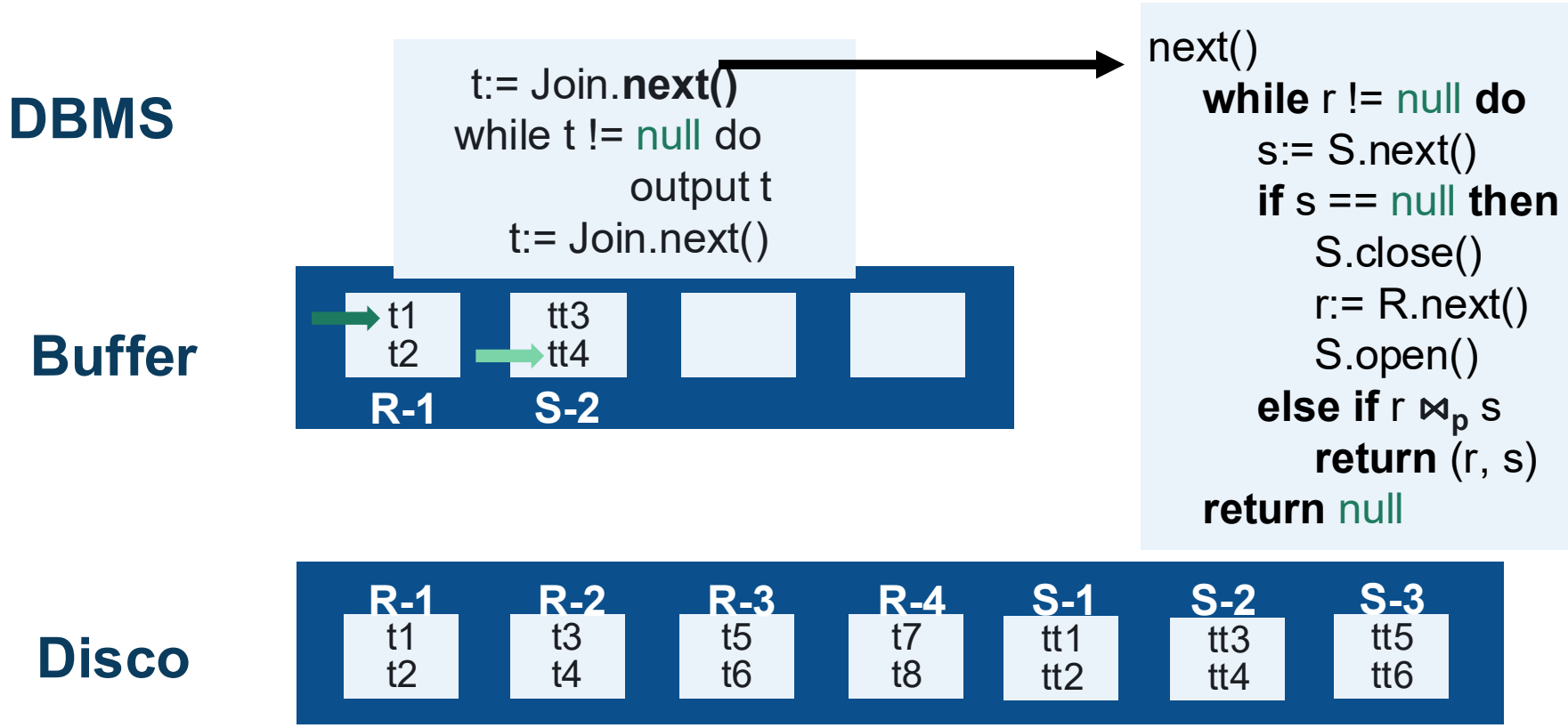
Nested Loop Join



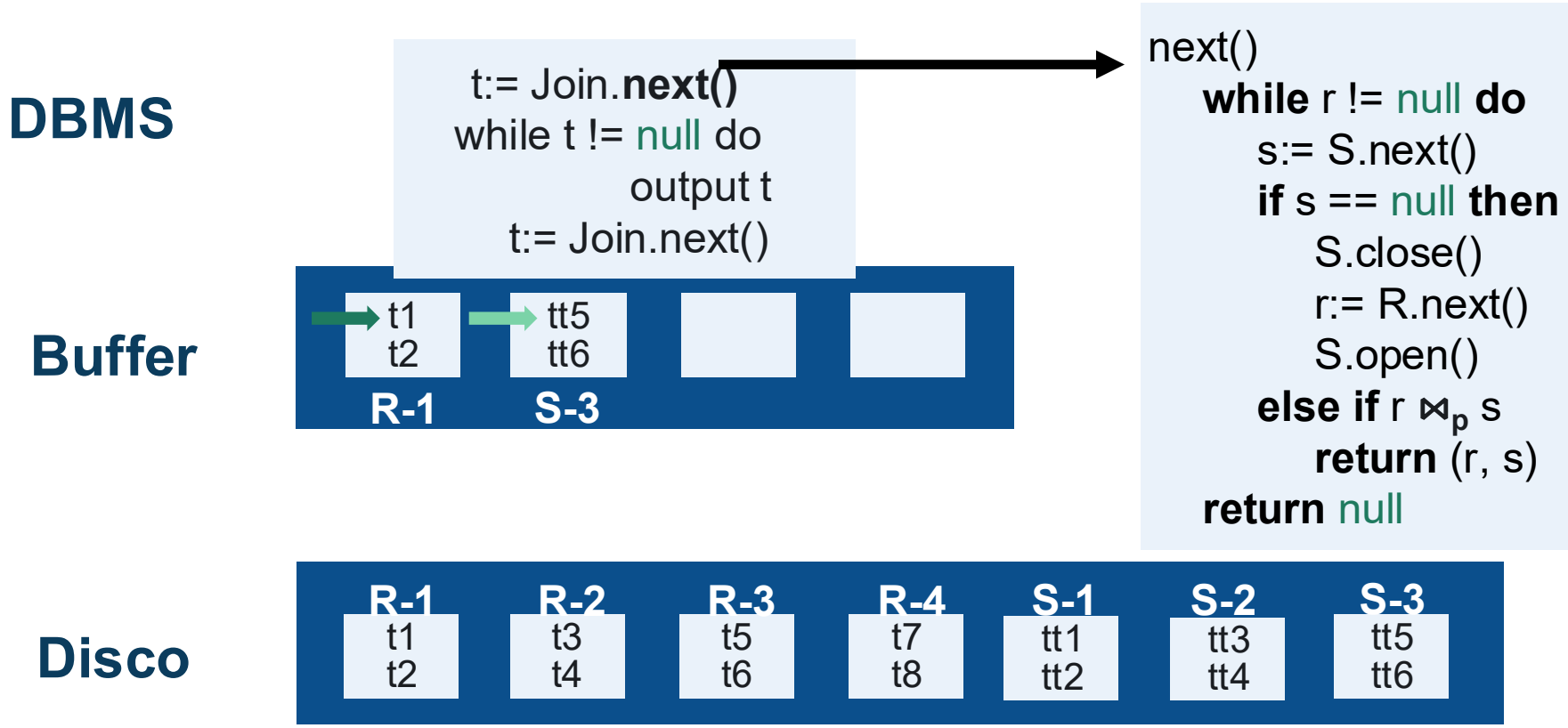
Nested Loop Join



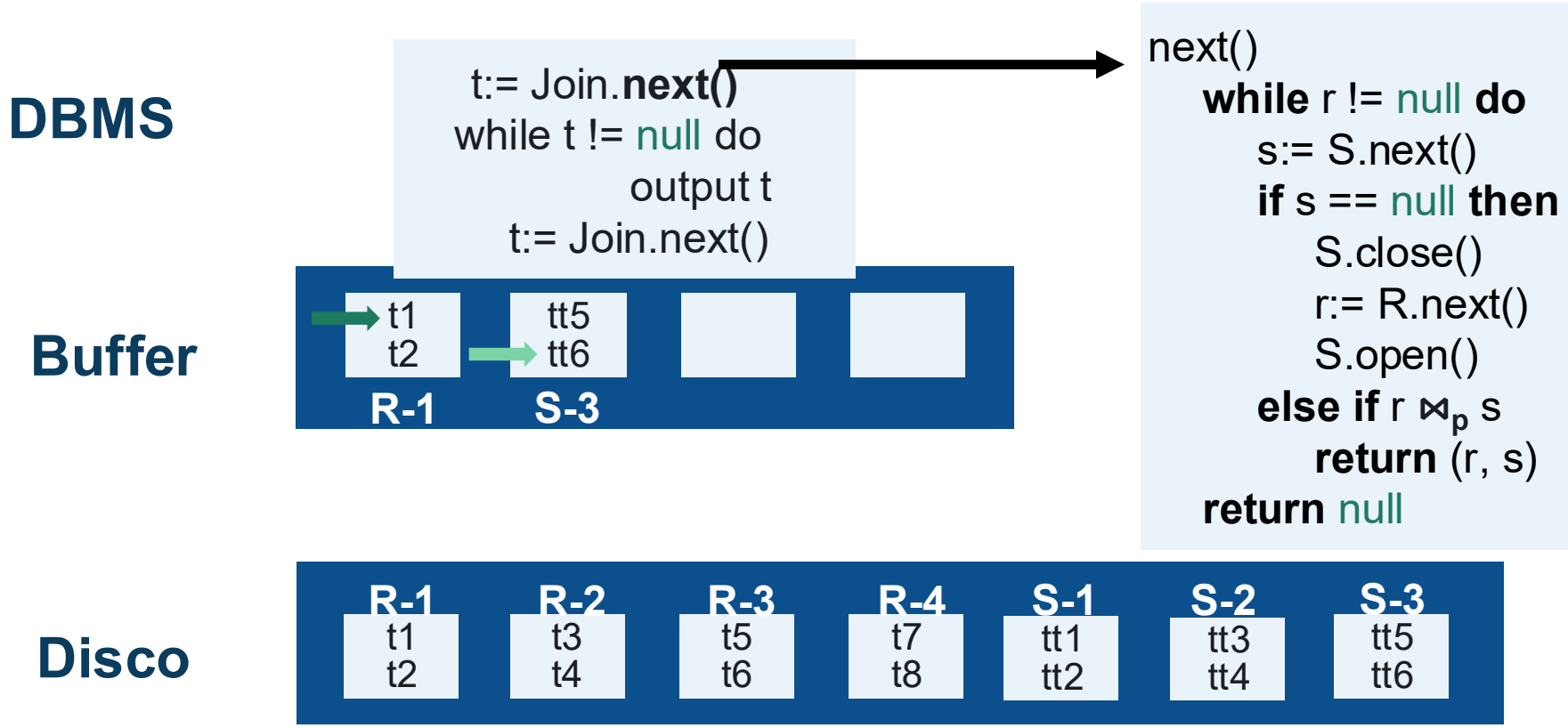
Nested Loop Join



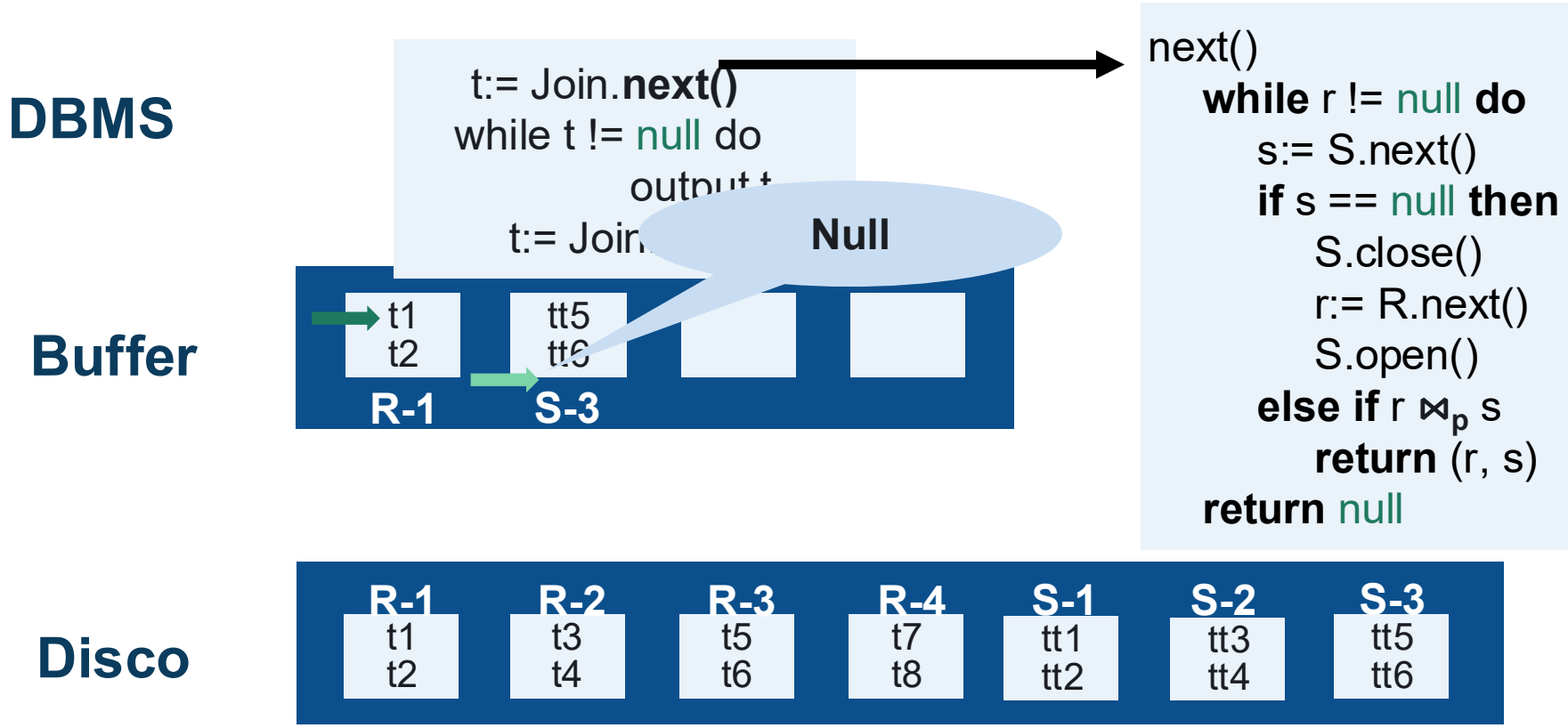
Nested Loop Join



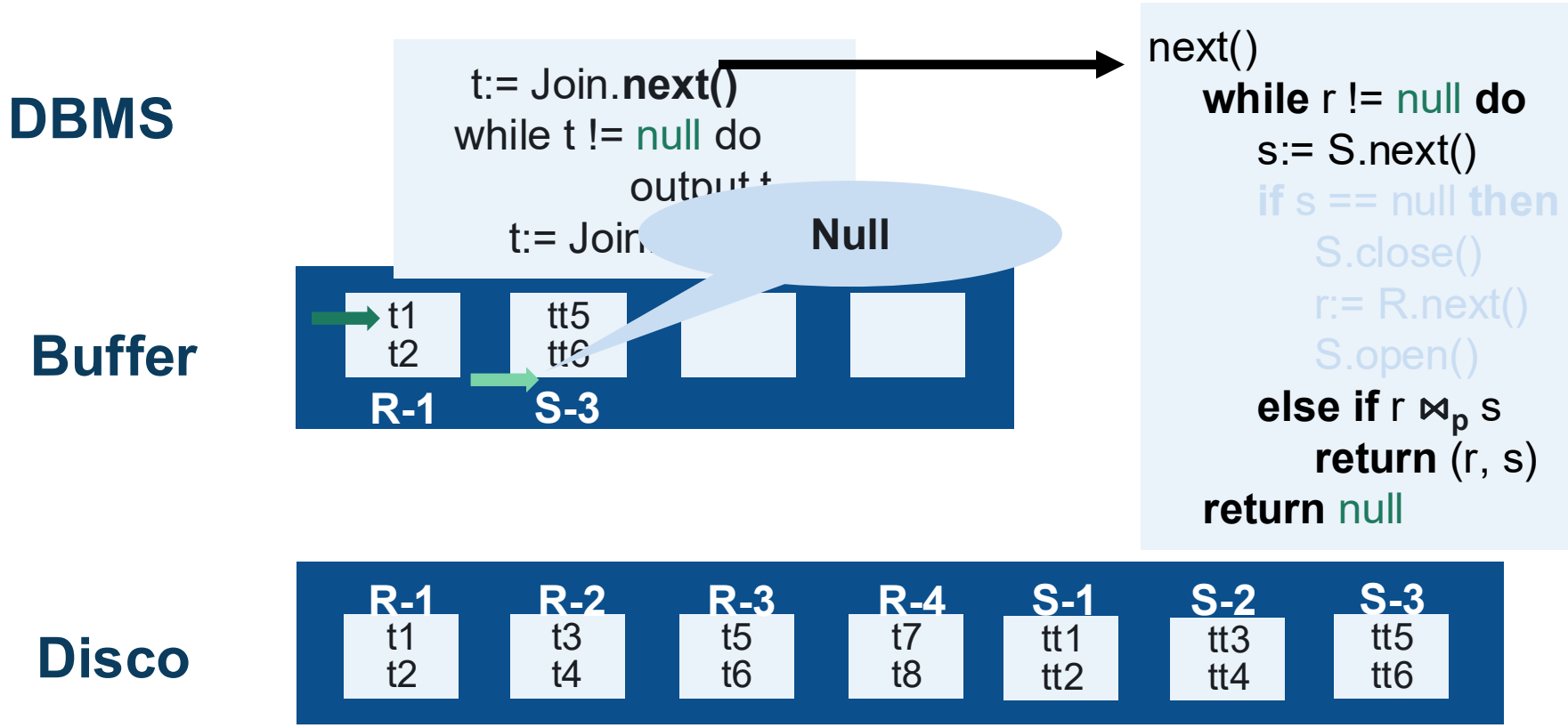
Nested Loop Join



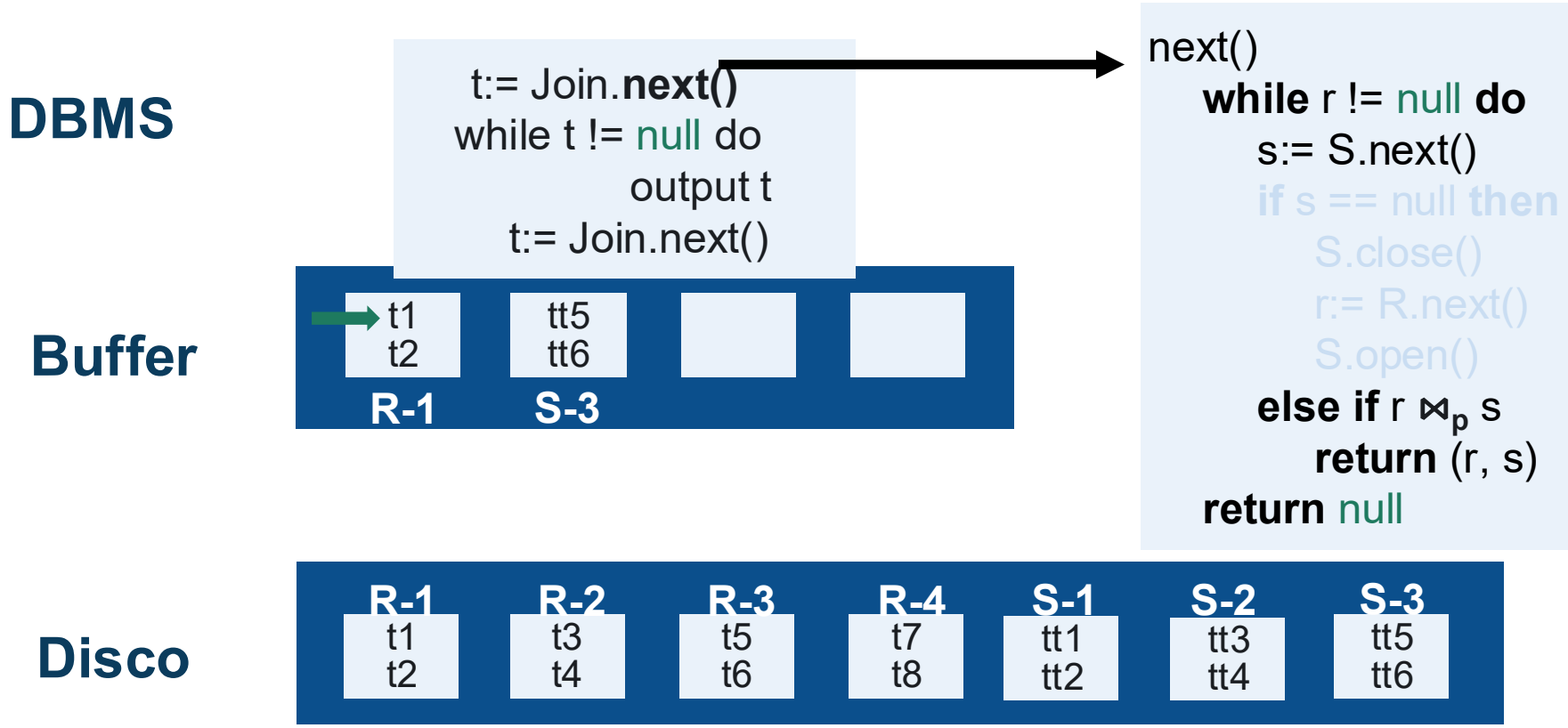
Nested Loop Join



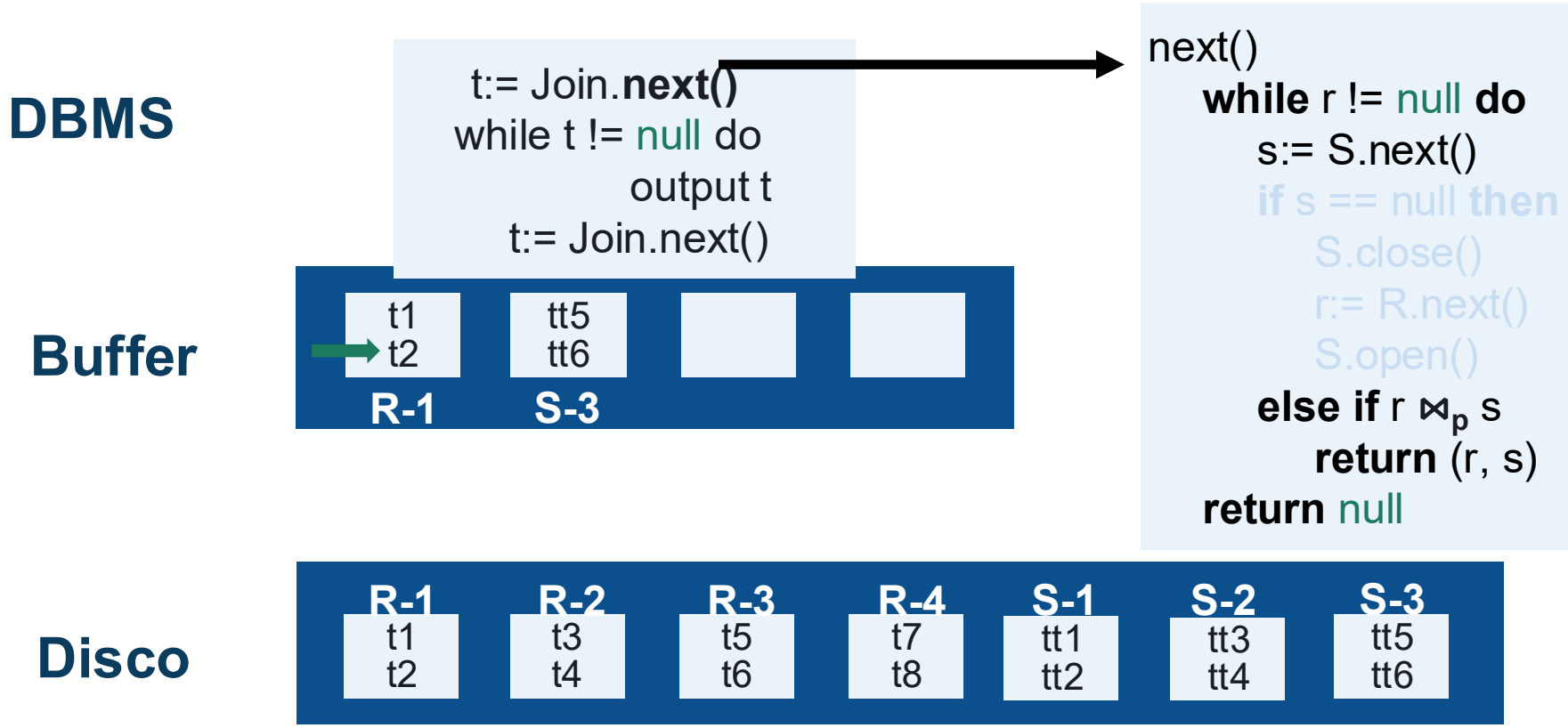
Nested Loop Join



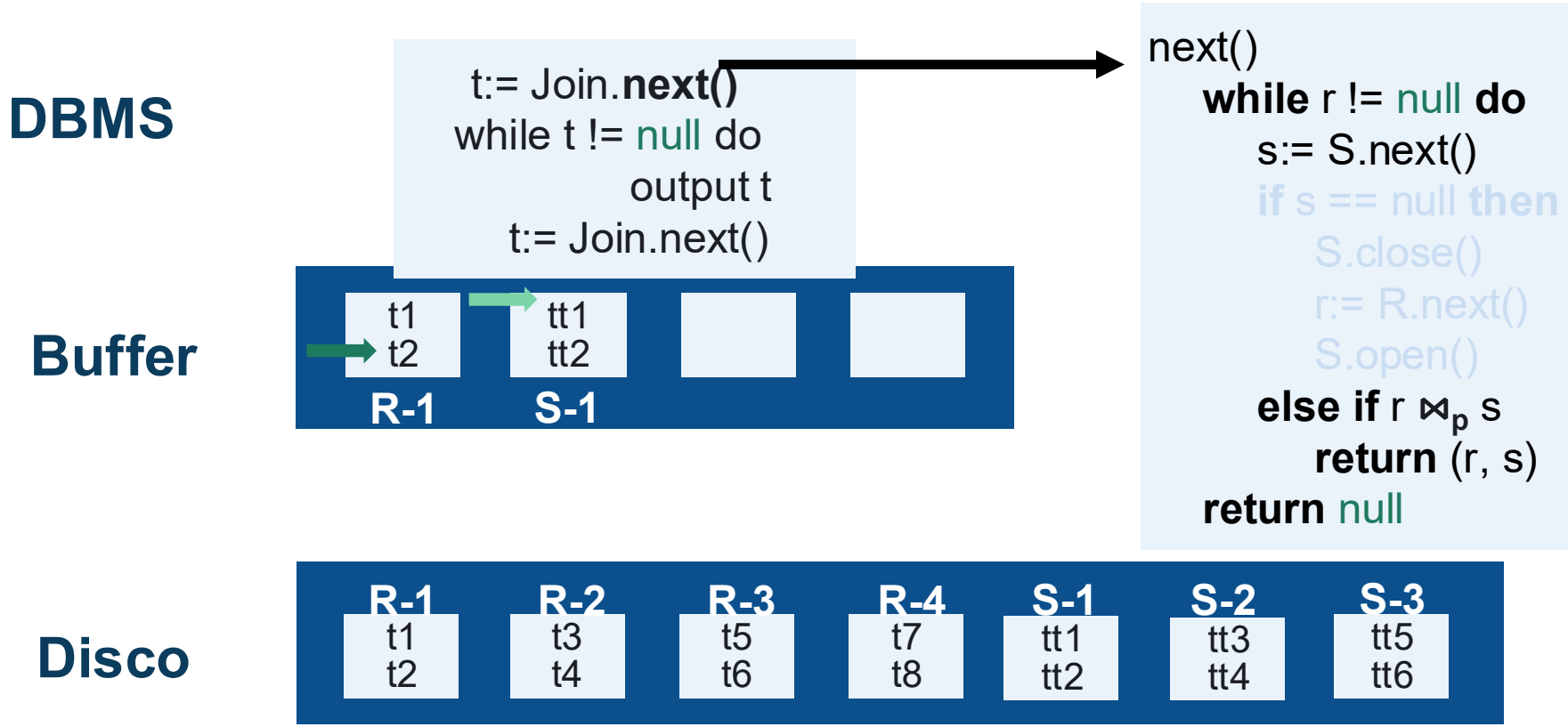
Nested Loop Join



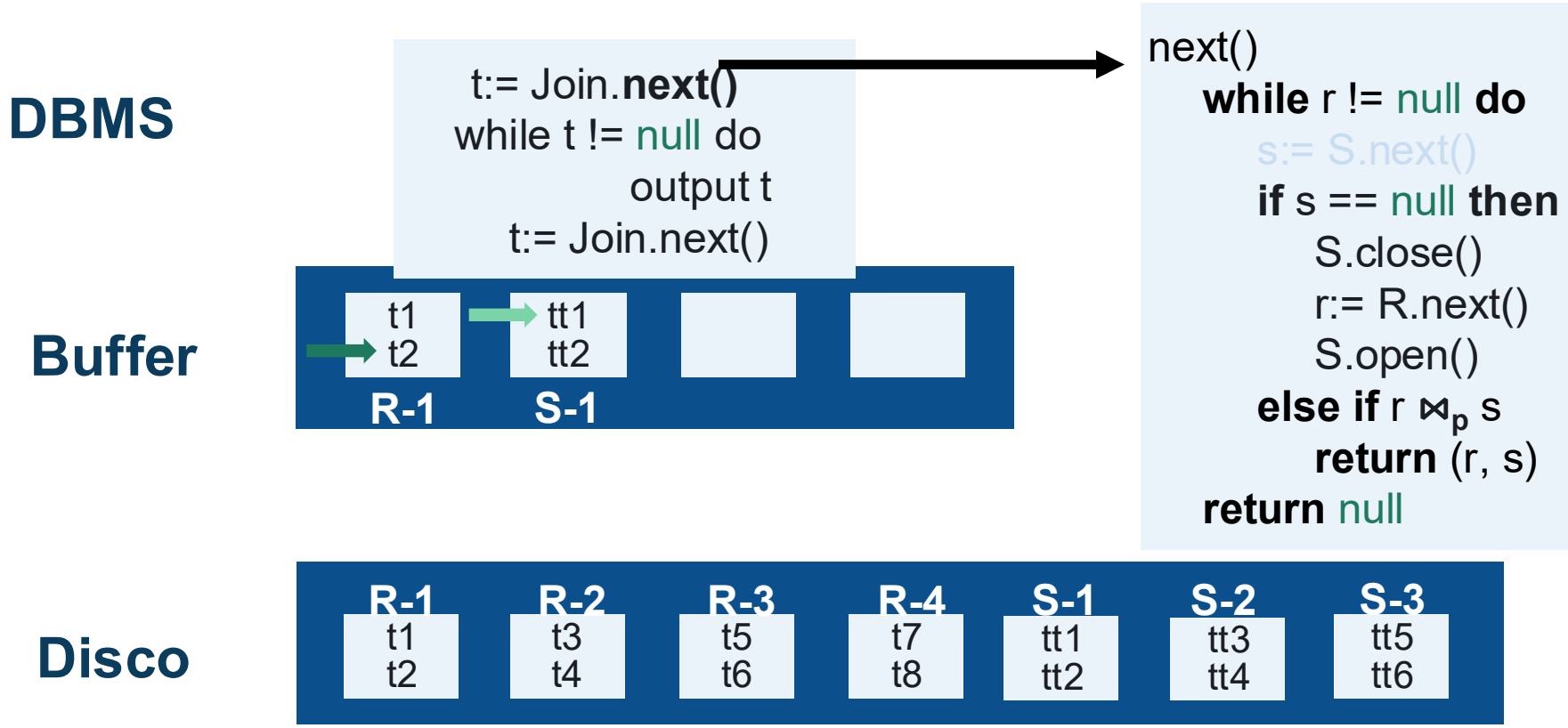
Nested Loop Join



Nested Loop Join



Nested Loop Join



Nested Loop Join

Es una implementación directa basada en un ciclo (loop).

Por cada tupla de **R** se debe leer **S** completa, además de leer **R** completa una vez.

Costo en I/O:

$$Costo(R) + Tuplas(R) \cdot Costo(S)$$

Nested Loop Join

Ejemplo de costo

- R y S son tablas de 16 MB; cada página es de 8 KB y las tuplas son de 300 bytes.
- Cada relación tiene 2.048 páginas y aproximadamente 55.000 tuplas.
- Si el costo de un I/O es 0,1 ms, el join tarda:



3,1 horas

tiempo estimado del join



¿Cómo optimizarlo? Con **Block Nested Loop Join**.

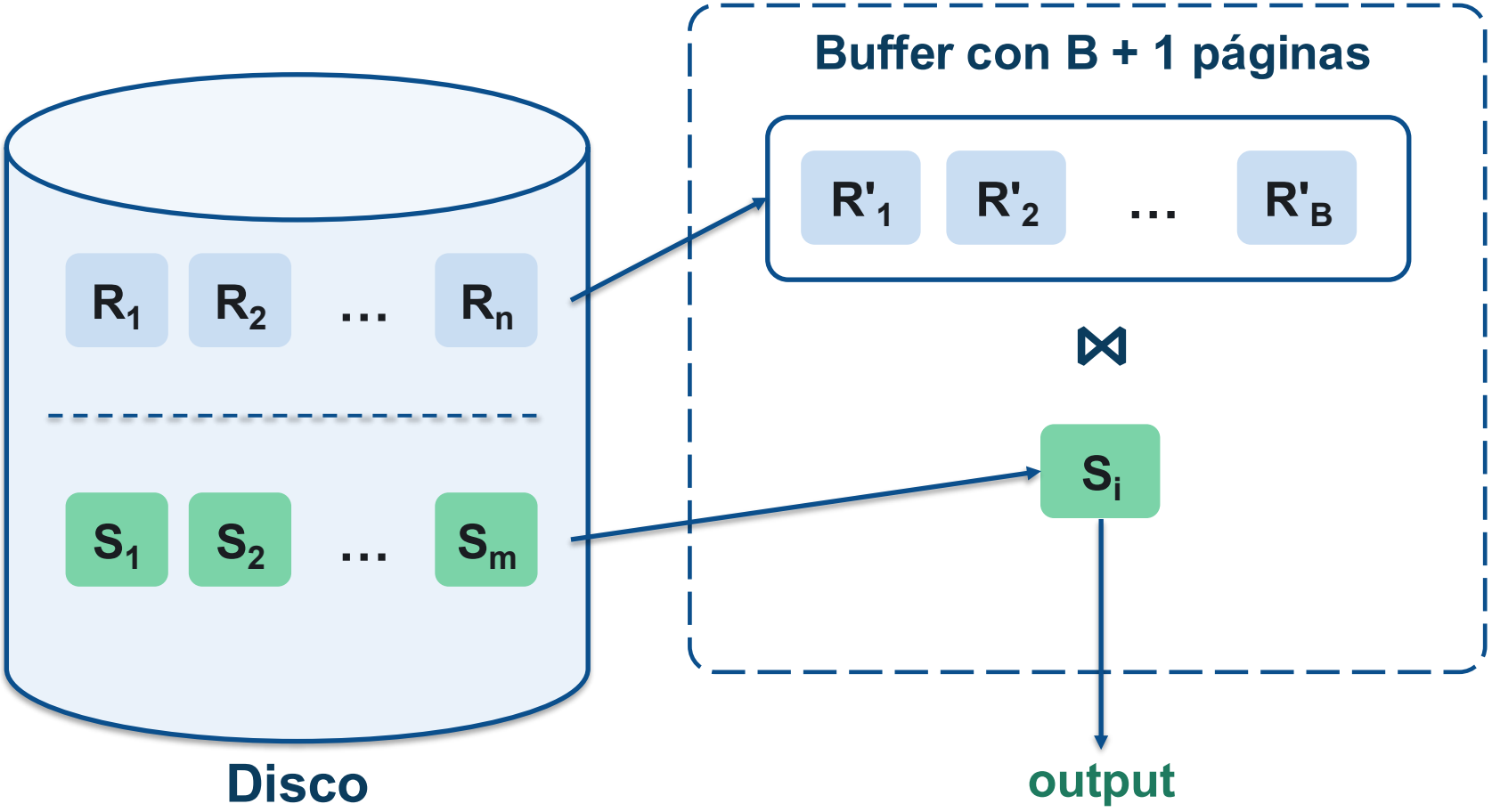


Block Nested Loop Join

Uso eficiente del buffer

Block Nested Loop Join

Se aprovecha mejor el buffer.



Block Nested Loop Join

Se desea calcular el join entre **R** y **S** cuando se satisface un predicado **p**.

```
open()
  R.open()
  fillBuffer()

close()
  R.close()
  S.close()
```

```
fillBuffer()
  Buff = empty
  r := R.next()
  while r != null do
    Buff = Buff union r
    if Buff.isFull() then
      break
    r := R.next()
  S.open()
  S.next()
```

Block Nested Loop Join

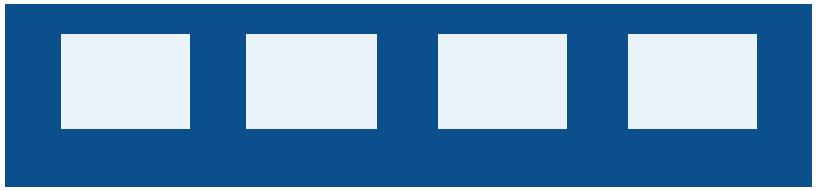
Se desea calcular el join entre **R** y **S** cuando se satisface un predicado **p**.

```
next()
  while Buff != empty do
    while s != null do
      r := Buffer.next()
      if r == null then
        Buffer.reset()
        s := S.next()
      else if (r, s) satisfacen p then
        return (r, s)
    fillBuffer()
  return null
```

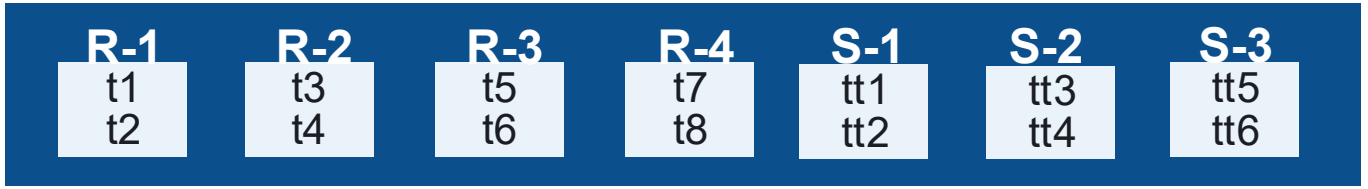
Block Nested Loop Join

DBMS $R \bowtie_p S$

Buffer



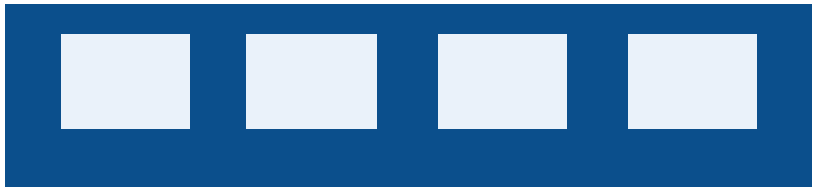
Disco



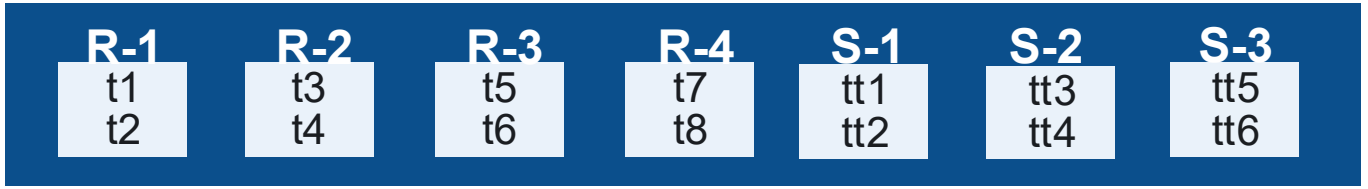
Block Nested Loop Join

DBMS $R \bowtie_p S$.open()

Buffer



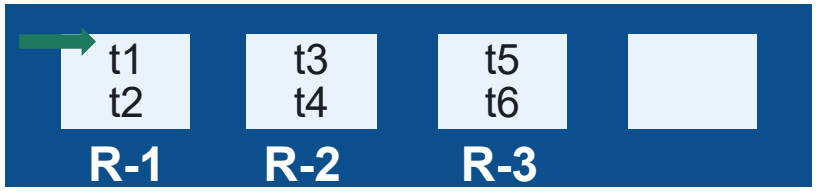
Disco



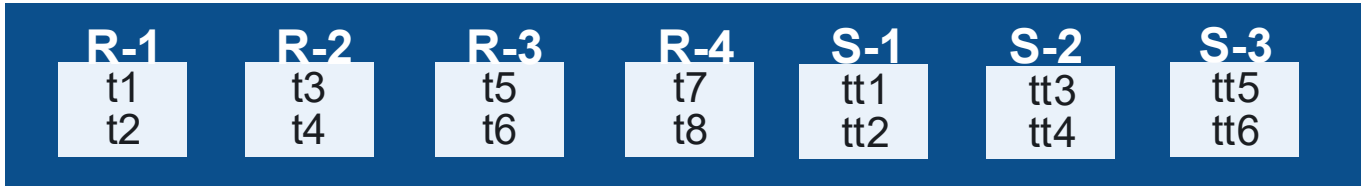
Block Nested Loop Join

DBMS $R \bowtie_p S$.open()

Buffer



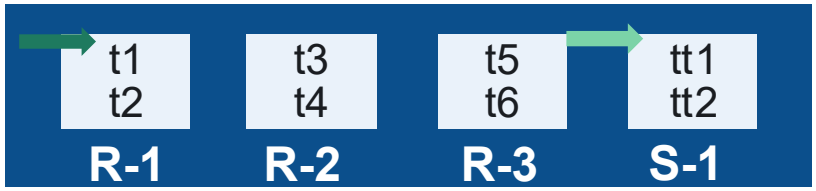
Disco



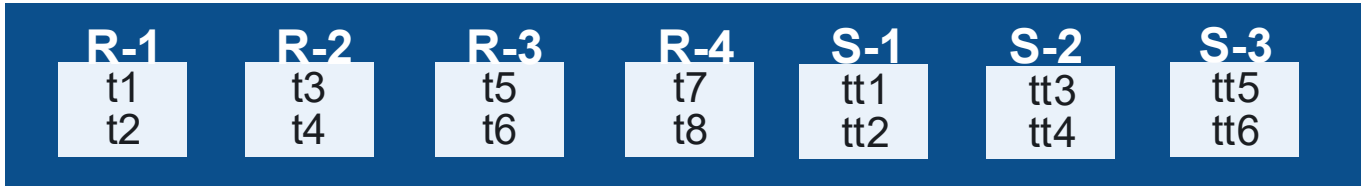
Block Nested Loop Join

DBMS $R \bowtie_p S$.open()

Buffer



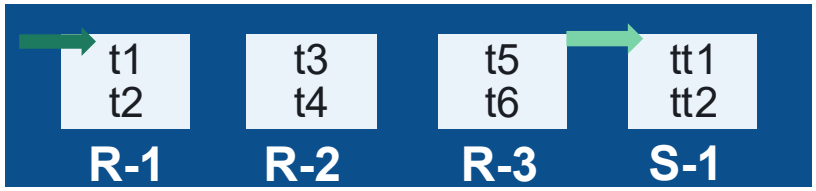
Disco



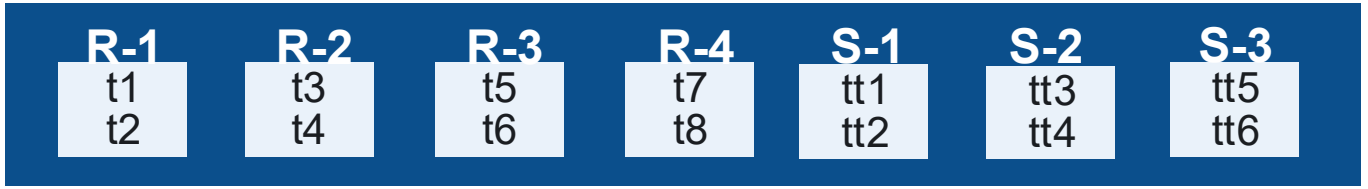
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



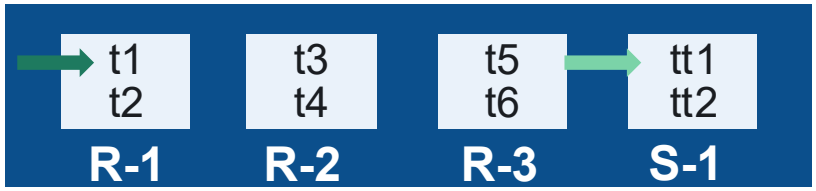
Disco



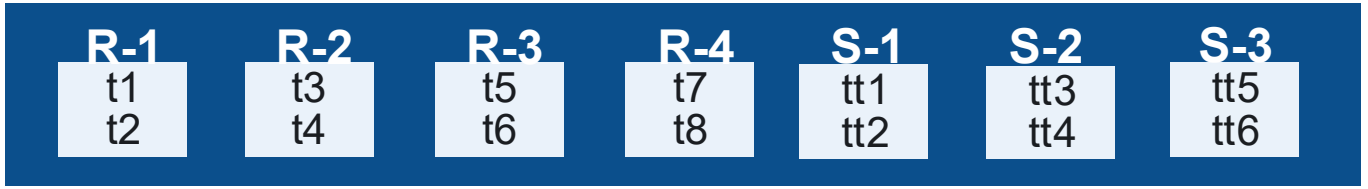
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



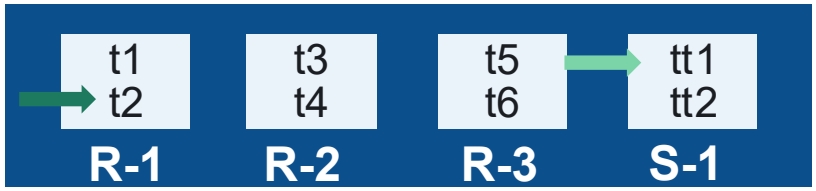
Disco



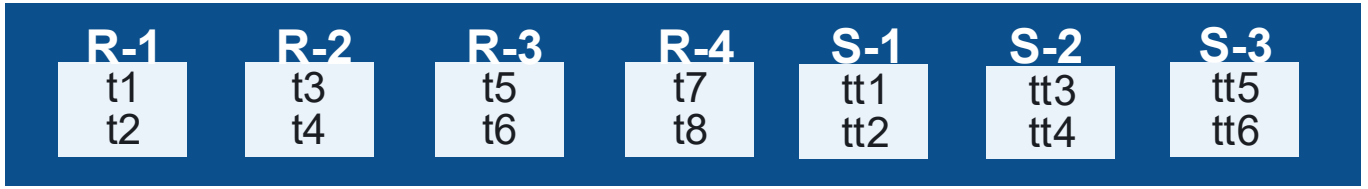
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



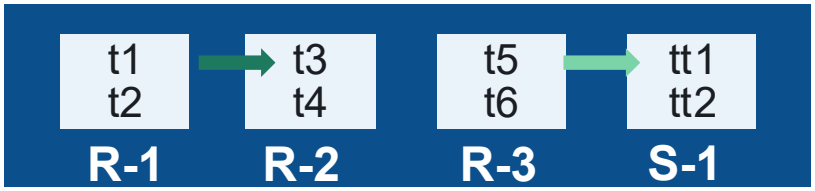
Disco



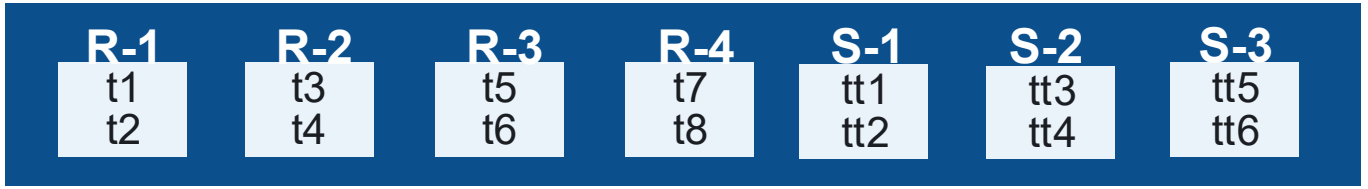
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



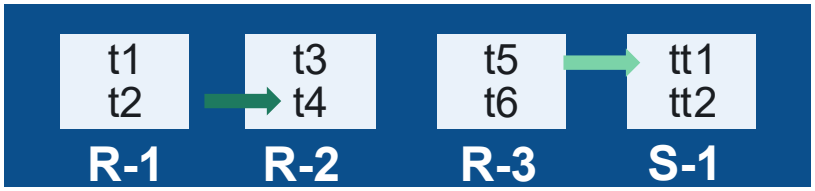
Disco



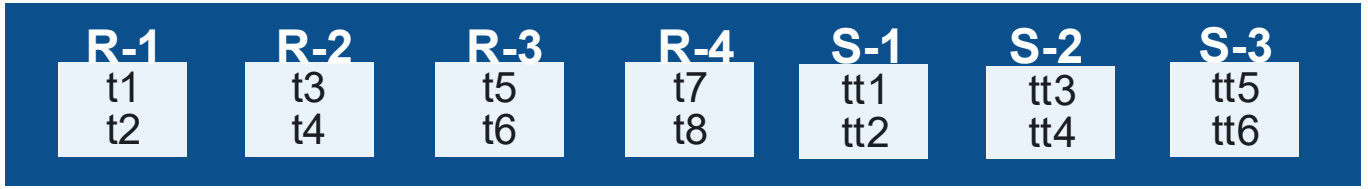
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



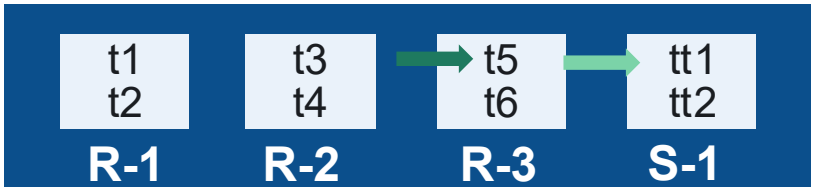
Disco



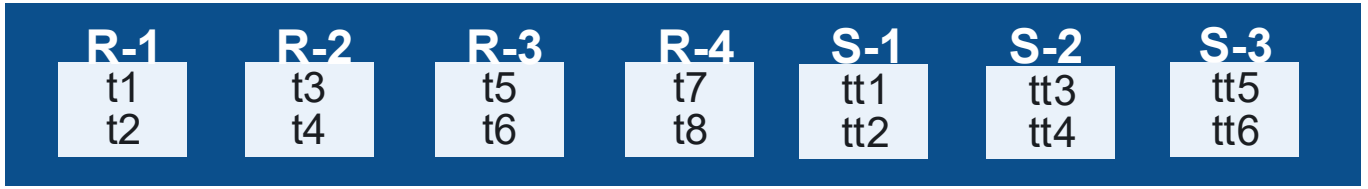
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



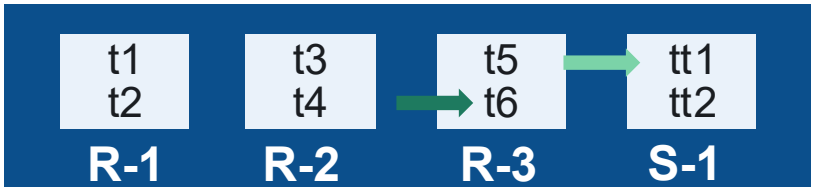
Disco



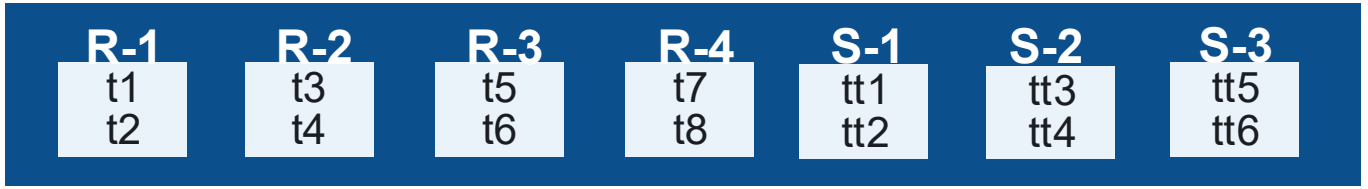
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer

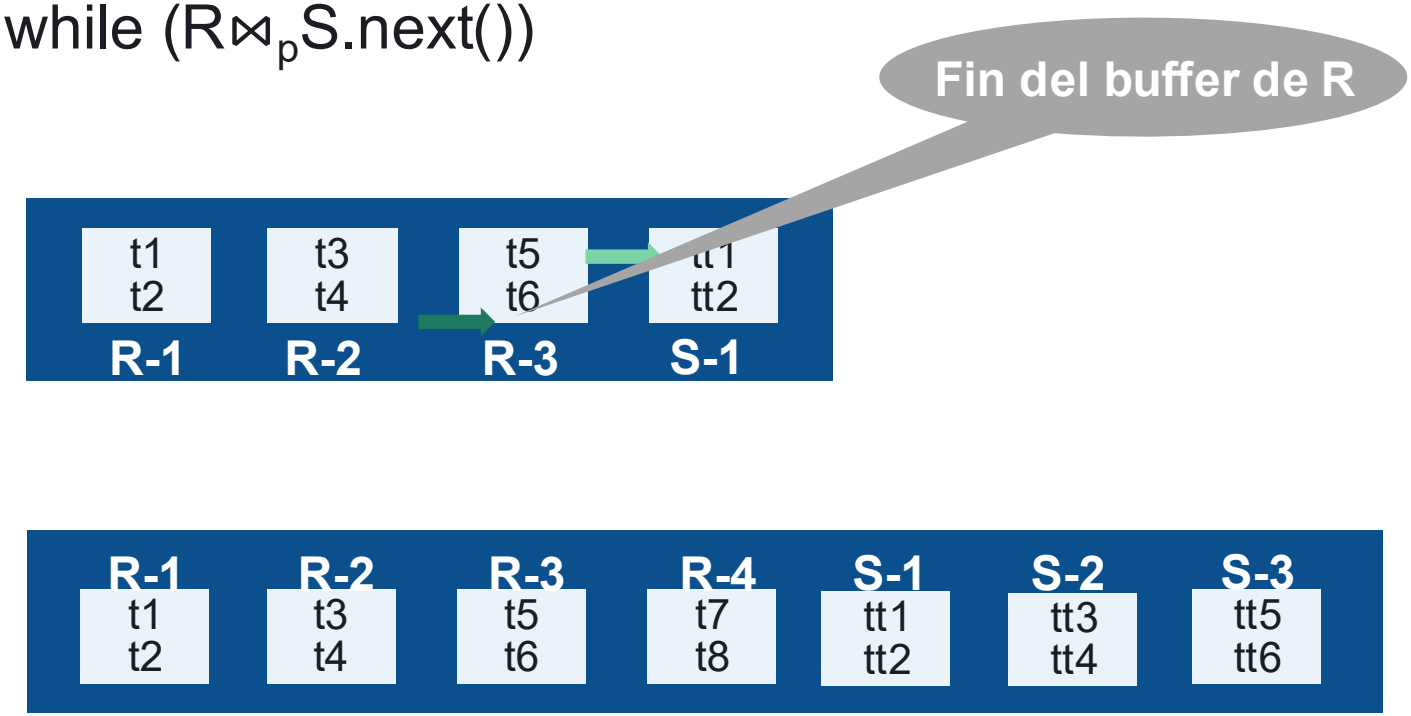


Disco



Block Nested Loop Join

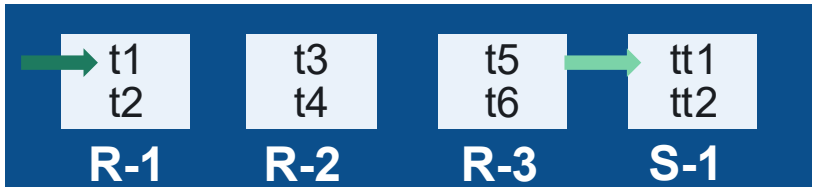
DBMS while ($R \bowtie_p S$.next())



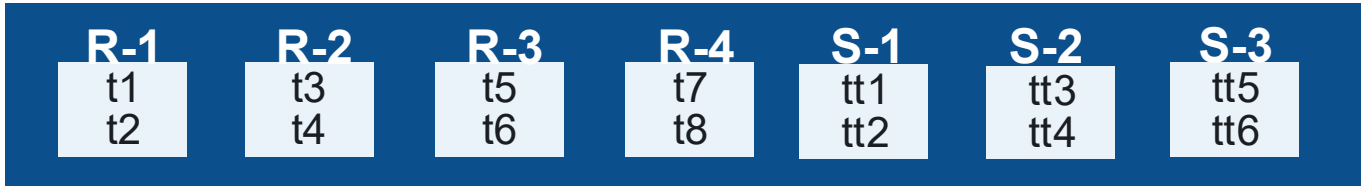
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



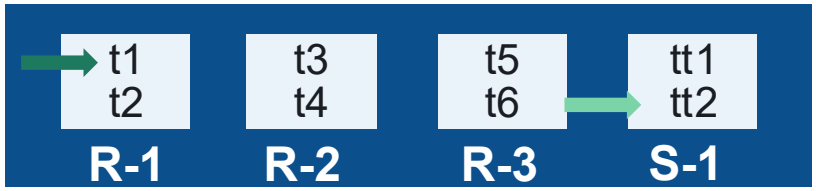
Disco



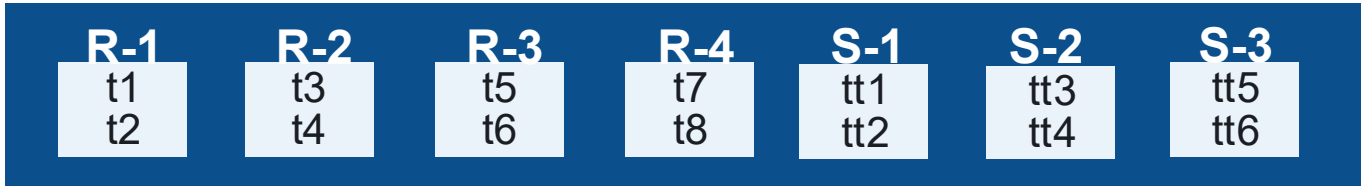
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



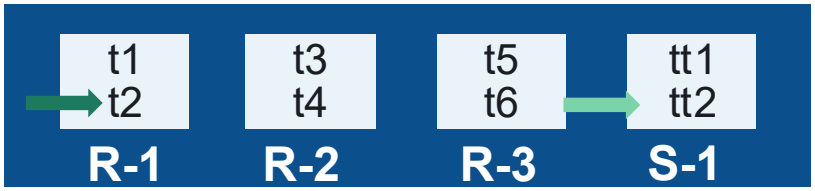
Disco



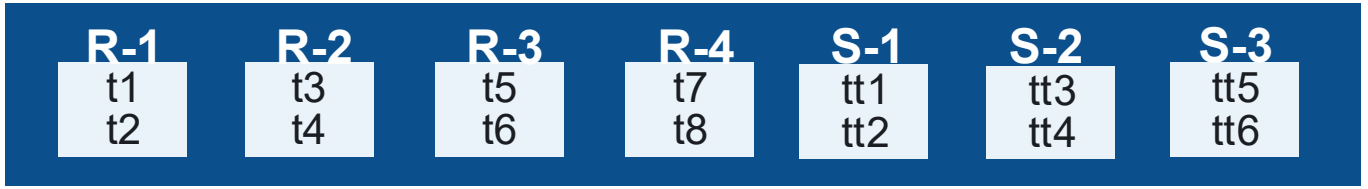
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



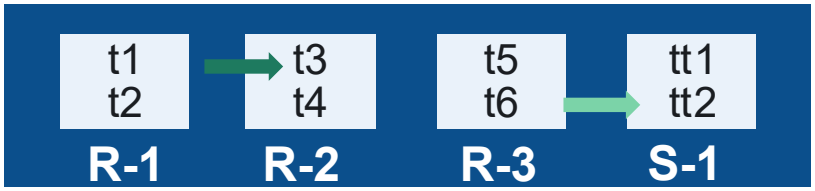
Disco



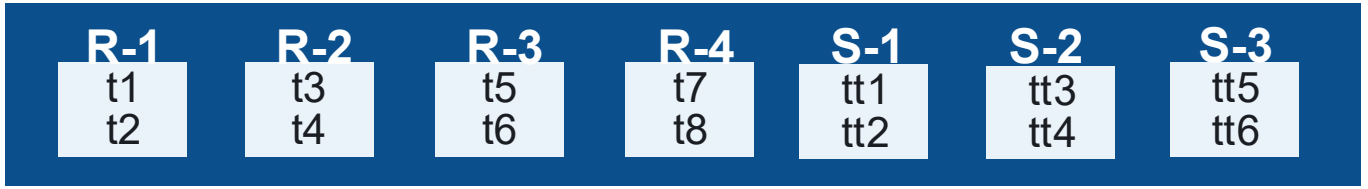
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



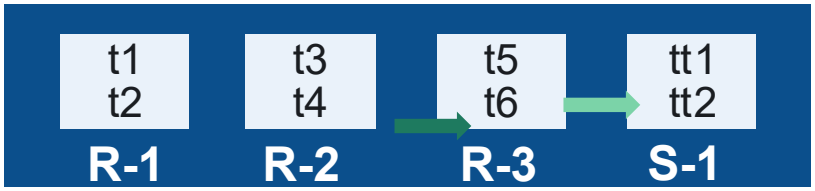
Disco



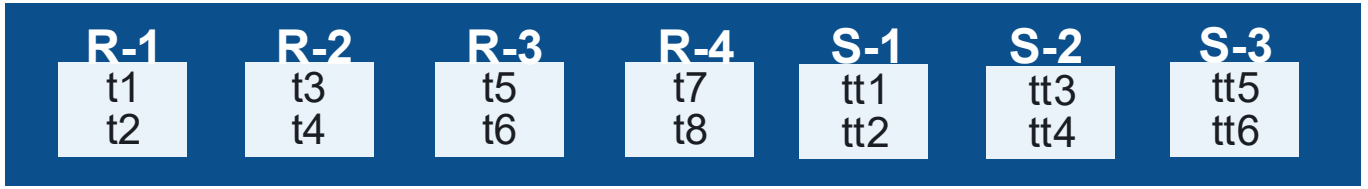
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



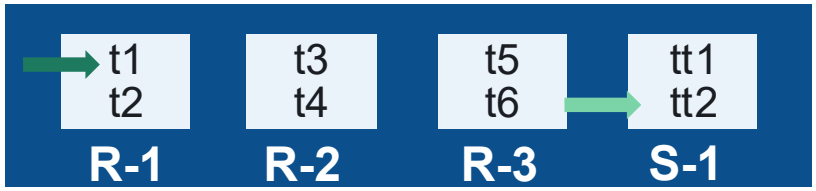
Disco



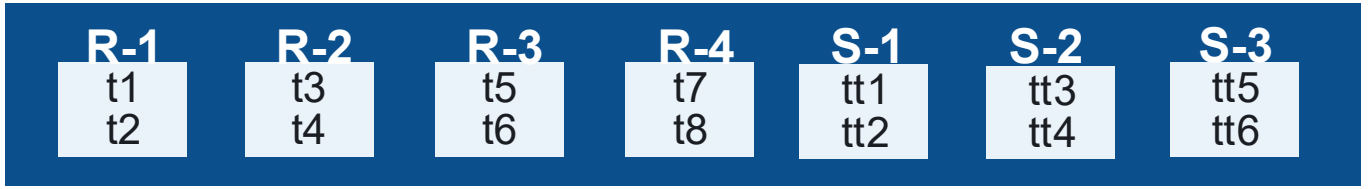
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



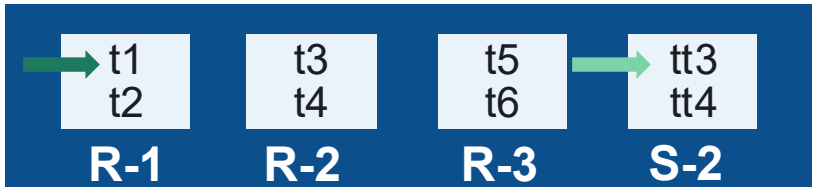
Disco



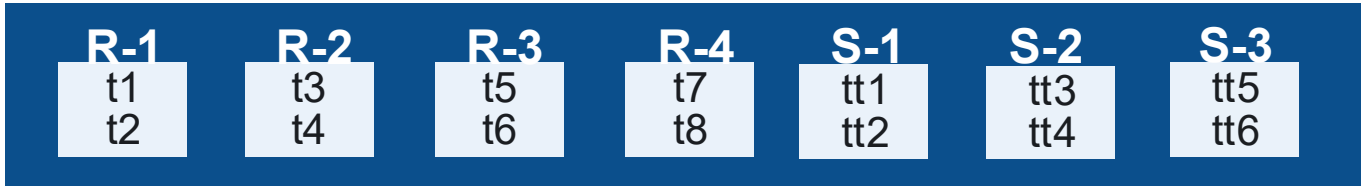
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



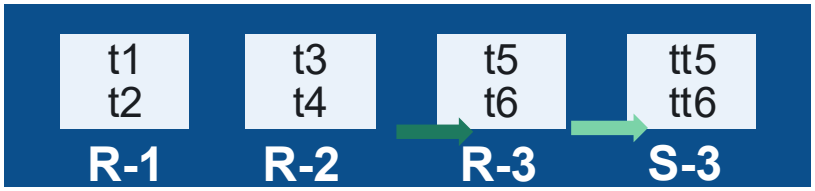
Disco



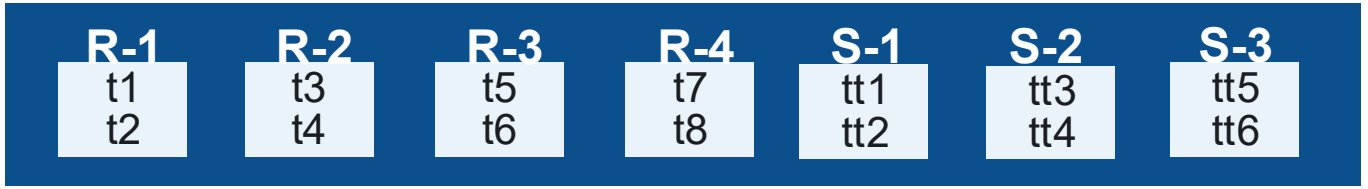
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



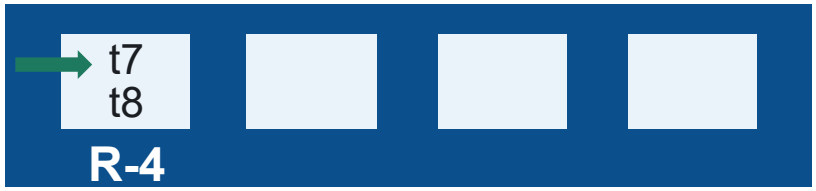
Disco



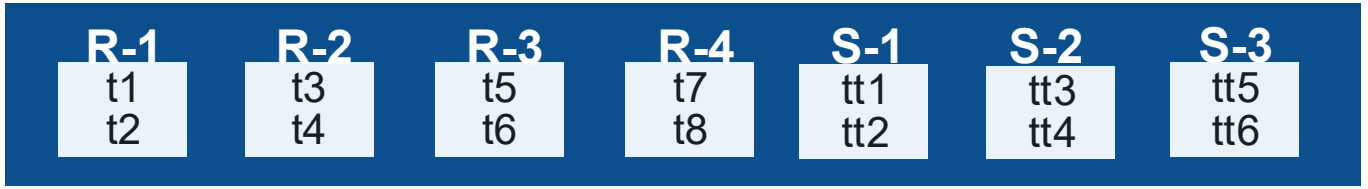
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



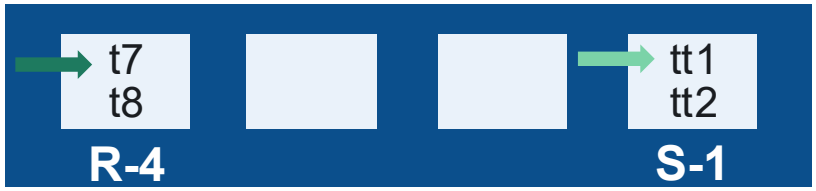
Disco



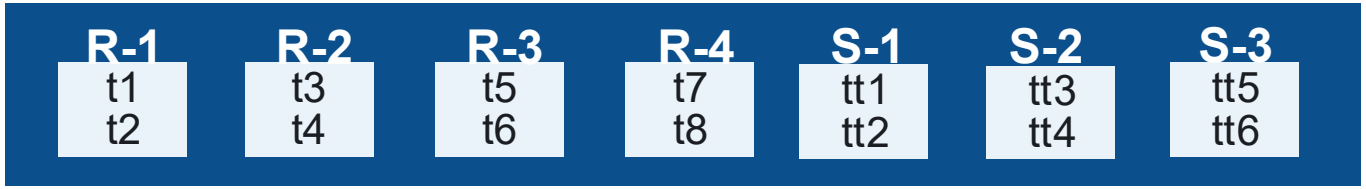
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



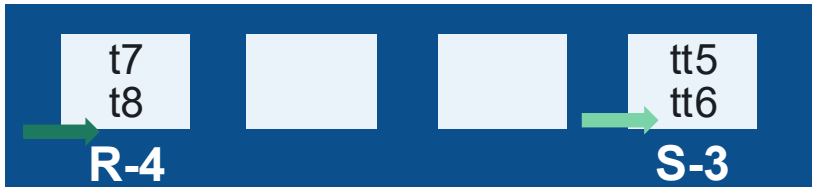
Disco



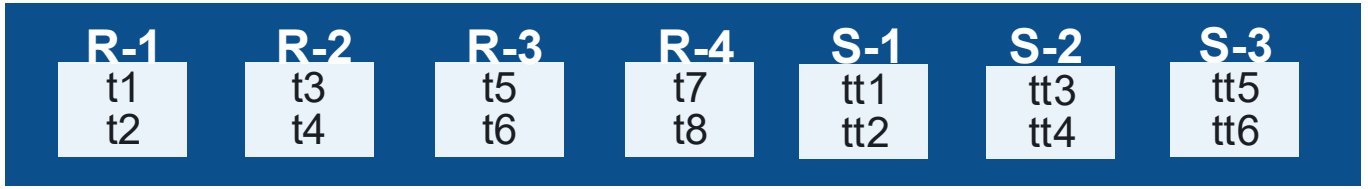
Block Nested Loop Join

DBMS while ($R \bowtie_p S$.next())

Buffer



Disco



Block Nested Loop Join



Se cargan muchas páginas de **R** al buffer.



Por cada llenado del buffer, se recorre **S** completa una vez.

Costo en I/O:

$$Costo(R) + (Páginas(R) / (Buffer - 2)) \cdot Costo(S)$$

Block Nested Loop Join

Ejemplo de costo

- R y S son tablas de 16 MB; cada página es de 8 KB y el buffer es de 1 MB.
- Cada relación tiene 2.048 páginas y en el buffer caben 128 páginas.
- Si el costo de un I/O es 0,1 ms, el join tarda:

3,4 segundos

Block Nested Loop Join

3,1 horas

Nested Loop Join
(diapositiva anterior)

Block Nested Loop Join

Sin embargo, existen algoritmos **mucho más eficientes**.



Hashing

Algoritmos basados en hashing.



Sorting

Algoritmos basados en ordenamiento.



Índices

Además, utilizan índices, como el árbol B+.

Sorting

- Los algoritmos de ordenamiento (sorting) son conocidos en programación.
- ¿Por qué estudiarlos otra vez?
- Para ordenar tuplas que exceden con creces el tamaño de la memoria RAM.
- En los DBMS se utiliza el algoritmo **External Merge Sort**.
- Antes de ello, se revisarán **Merge** y **MergeSort**.



Merge y MergeSort

Ordenamiento por mezcla

Merge

Mezcla de secuencias ordenadas

Se propone el siguiente algoritmo para combinar dos secuencias ordenadas y formar una nueva secuencia **ordenada**:

input: Secuencias ordenadas A y B
output: Nueva secuencia ordenada C

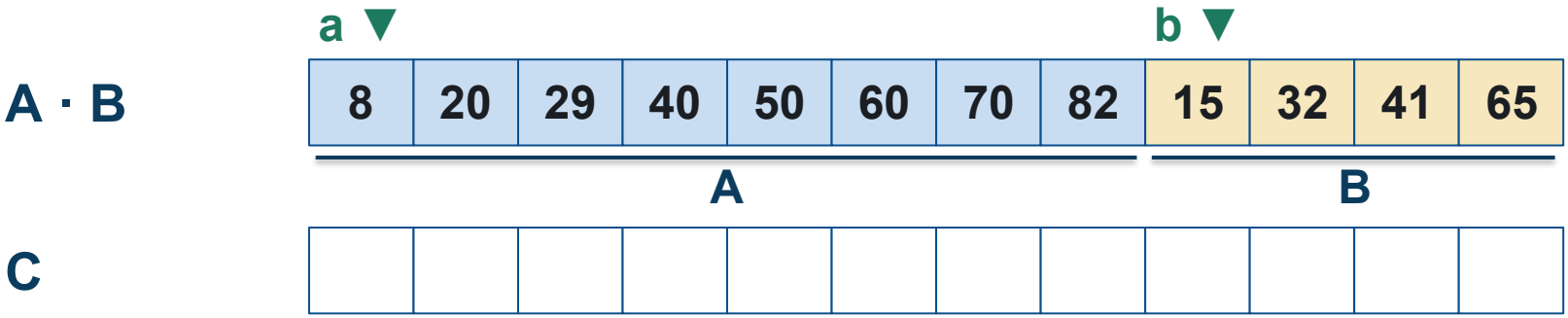
Merge(A, B):

1. Iniciar C vacía
2. Sean a y b los primeros elementos de A y B
3. Extraer de su secuencia respectiva el menor entre a y b
4. Insertar el elemento extraído al final de C
5. Si quedan elementos en A y en B, volver a la línea 2
6. Concatenar a C la secuencia que aún tenga elementos
7. **return C**

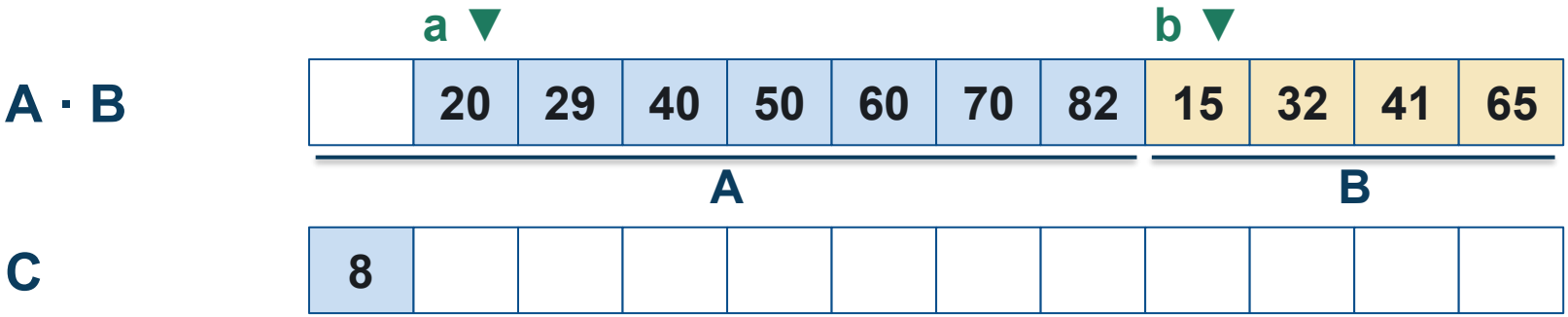
Merge

Ejemplo de ejecución

Estado inicial



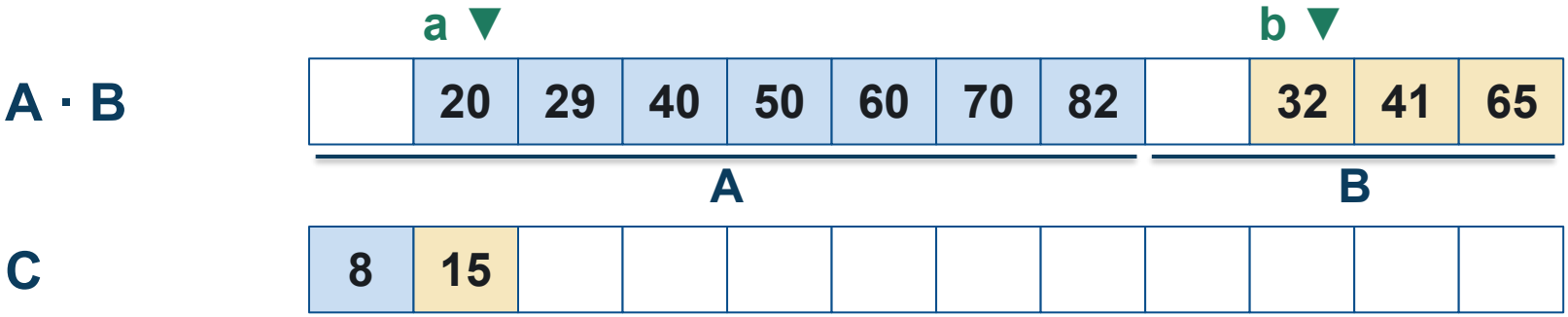
Estado luego de la primera iteración



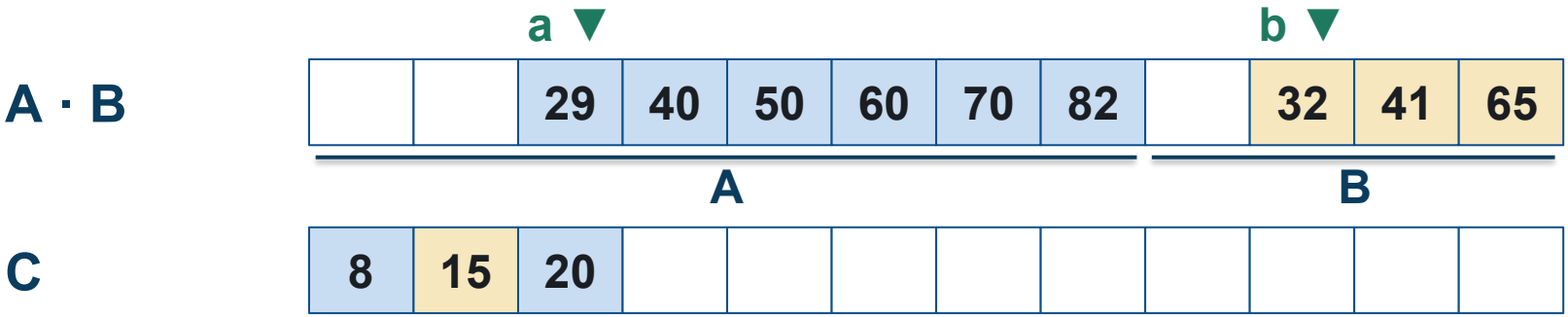
Merge

Ejemplo de ejecución

Estado luego de la segunda iteración



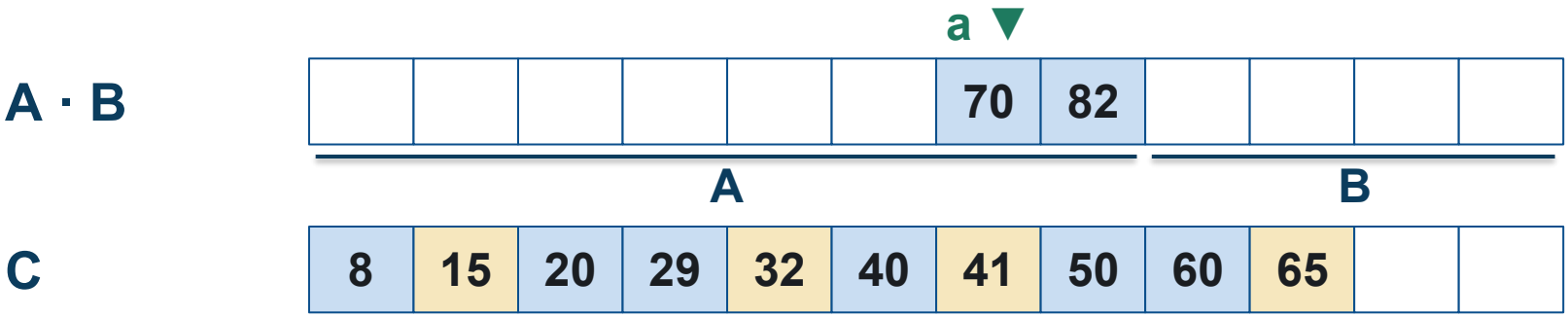
Estado luego de la tercera iteración



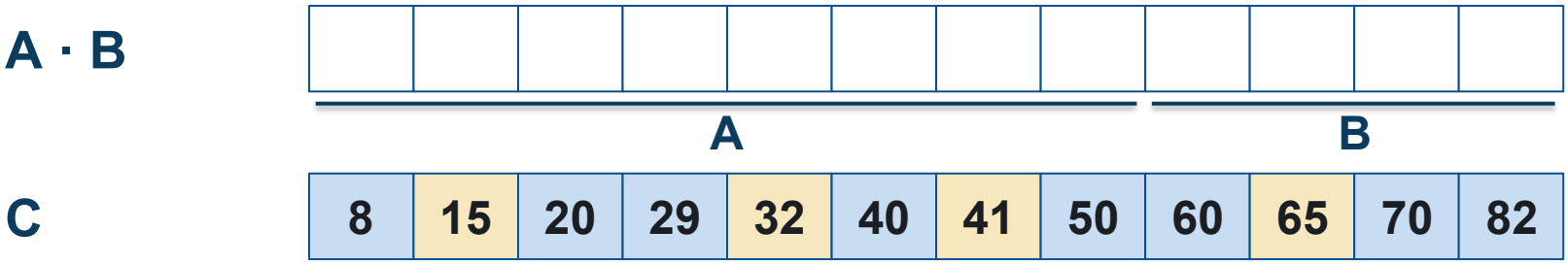
Merge

Ejemplo de ejecución

Estado luego de insertar en C el último elemento de B



Estado luego de concatenar el resto de A



Dividir para conquistar

Para usar Merge en un algoritmo de ordenamiento se sigue la estrategia **dividir para conquistar**, que consta de los siguientes pasos:

1. **Dividir** el problema original en dos (o más) subproblemas del mismo tipo.
2. Resolver **recursivamente** cada subproblema.
3. Encontrar la solución al problema original **combinando** las soluciones de los subproblemas.



Los subproblemas son instancias más pequeñas del problema a resolver.

Dividir para conquistar y Merge

La estrategia dividir para conquistar puede aplicarse al problema de ordenamiento usando Merge.



¿En qué parte de dividir para conquistar se utiliza Merge?

La idea general define un nuevo algoritmo, denominado **MergeSort**:

1. Dividir la secuencia original en dos subsecuencias.
2. Invocar recursivamente MergeSort sobre ambas subsecuencias.
3. Combinar las secuencias ordenadas resultantes mediante Merge.

El algoritmo MergeSort

A continuación se presenta el pseudocódigo del algoritmo recursivo MergeSort:

```
input:  Secuencia A
output: Secuencia ordenada B

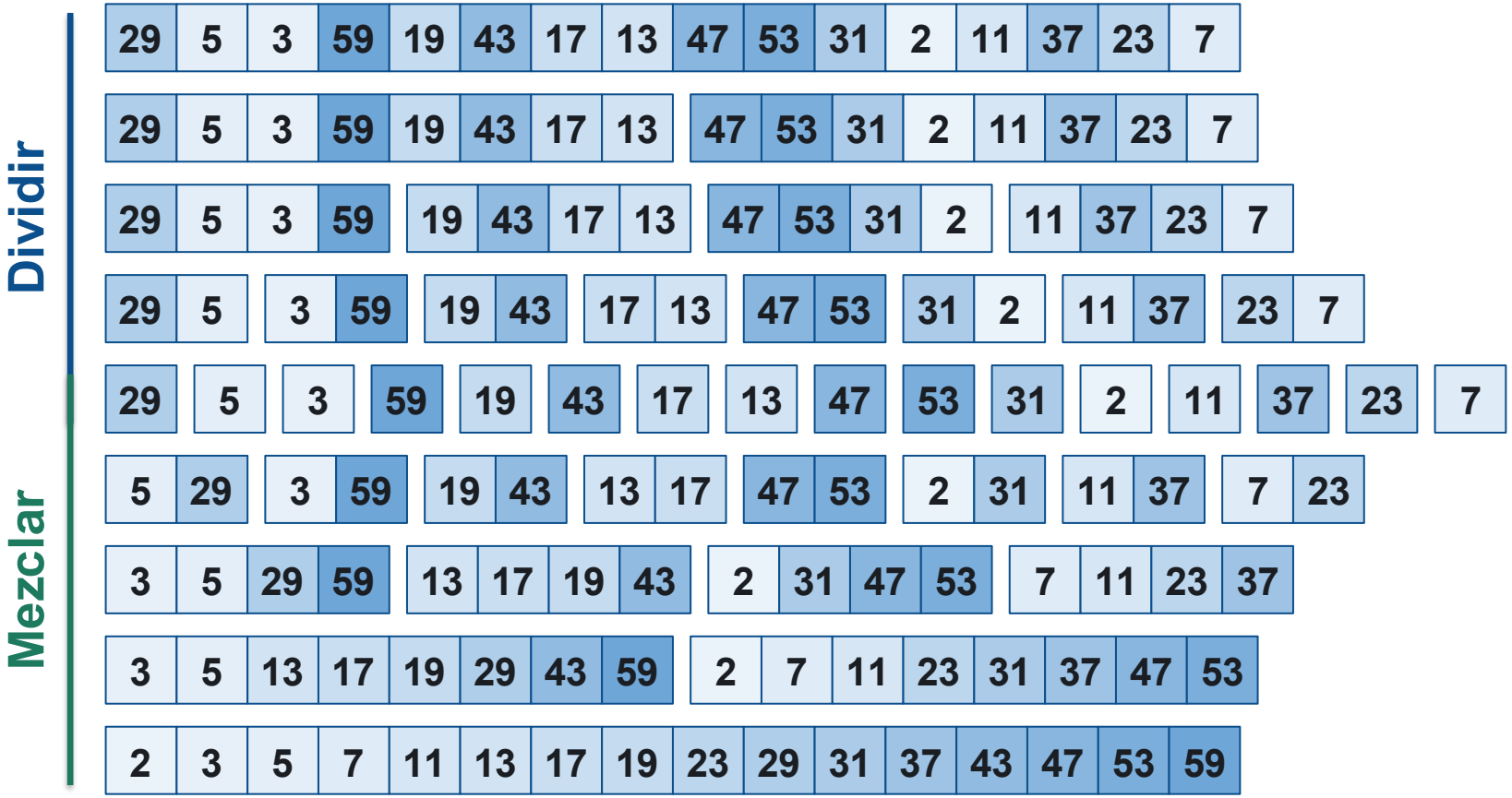
MergeSort(A):
1  if |A| = 1: return A
2  Dividir A en mitades A1 y A2
3  B1 ← MergeSort(A1)
4  B2 ← MergeSort(A2)
5  B ← Merge(B1, B2)
6  return B
```



Algoritmo recursivo

MergeSort

Ejemplo de ejecución





External Merge Sort

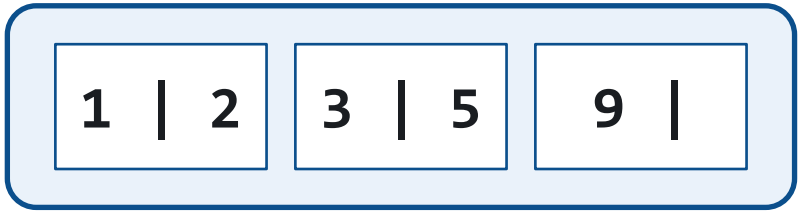
Ordenamiento en disco

External Merge Sort

En los DBMS se utiliza el algoritmo **External Merge Sort**.

Se denomina **run** a una secuencia de páginas que contiene un conjunto ordenado de tuplas.

El algoritmo funciona por **fases**.



Ejemplo de run: 3 páginas con tuplas ordenadas

External Merge Sort



Fase 0

Se crean los runs iniciales.

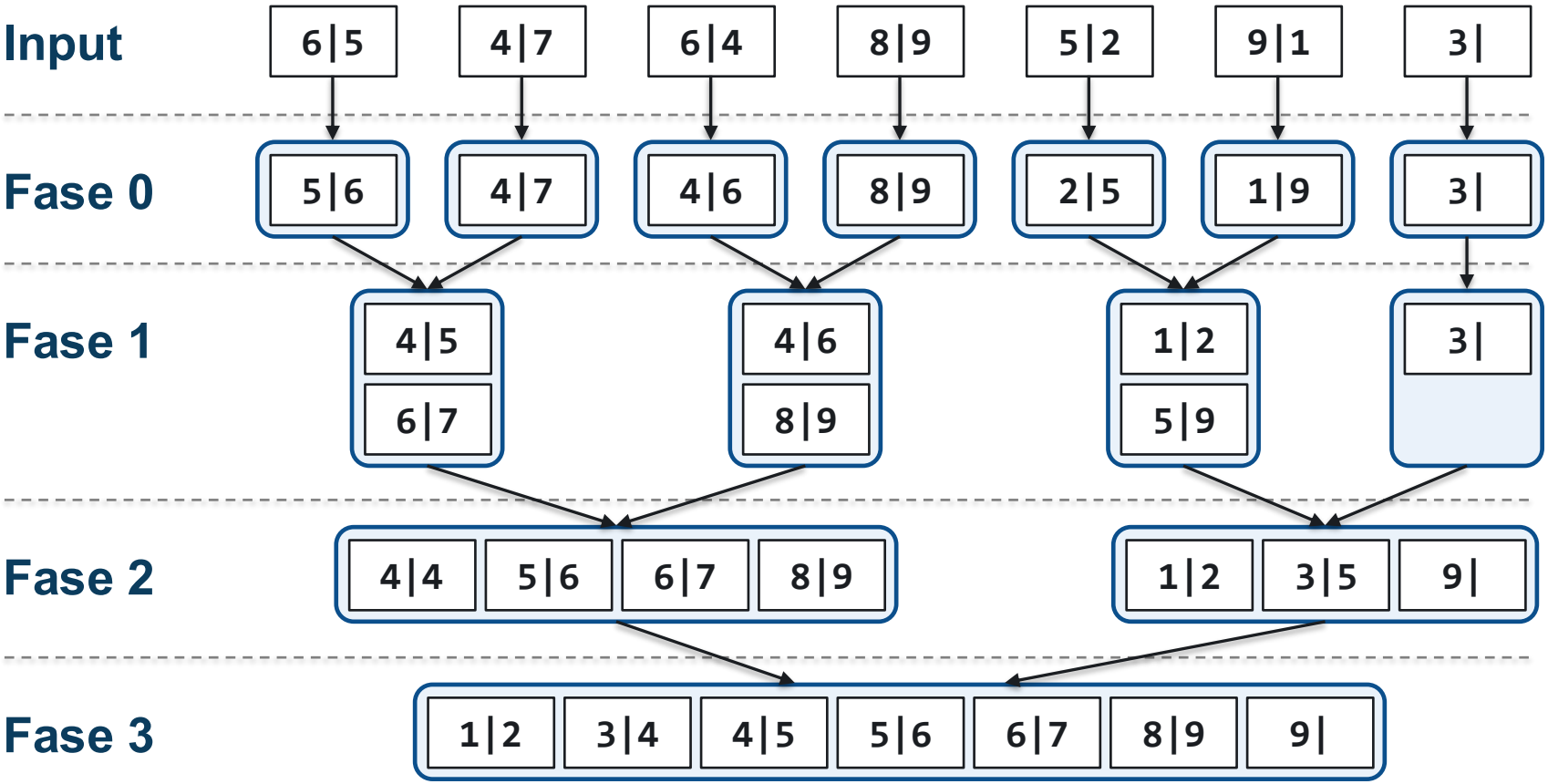


Fase i

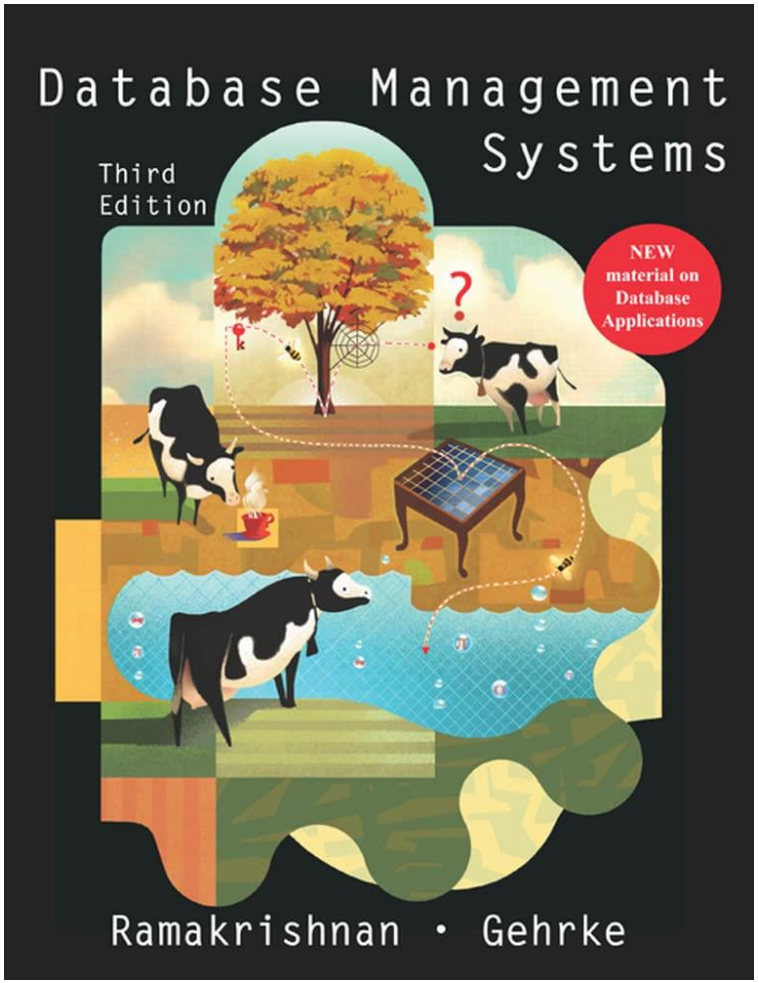
1. Se traen los runs a memoria.
2. Se realiza el merge de cada par de runs.
3. Se almacena el nuevo run en disco (es decir, se materializan los resultados intermedios).

External Merge Sort

Ejemplo de ejecución



Cada página contiene dos tuplas; cada recuadro azul es un run.



Bibliografía

Ramakrishnan, R. y Gehrke, J.

Database Management Systems, 3.ª edición.

Capítulo: Evaluación de consultas

Biblioteca UC (acceso digital)